CSCI-GA 2250-002

Operating Systems

Course Notes

Fall 2026

Instructor: Dr. Yang Tang

Notes by You Li

Acknowledgements

These notes follow the Fall 2026 lectures of CSCI-GA 2250-002 (Operating Systems) at the Courant Institute, taught by Dr. Yang Tang. Each chapter follows one slide deck: Chapter 1 covers *1 — Introduction*, and Chapter 2 covers *2 — Process Management*.

Notation. Code, commands, and file names are set in `typewriter`. A call such as `fork()` refers to the function; `man 2 fork` refers to section 2 of the manual (system calls), and section 3 holds library functions. Shell sessions begin with the prompt `$`. PIDs in the examples (2436, 2437, . . .) follow the slides. Figures captioned "after slide n" redraw the corresponding slide; in timelines, a solid line means running, a dashed line means blocked or stopped, and a grey line marks a zombie.

Contents

1 Introduction **5**

1.1 What Is an Operating System? . . . 6
1.1.1 Where the OS Fits In . . . 6
1.1.2 Example: What Happens When You Type `ls` . . . 6
1.1.3 Two Viewpoints . . . 7
1.2 Interacting with the OS . . . 8
1.2.1 Three Points of View . . . 8
1.2.2 Interrupts . . . 8
1.2.3 System Calls . . . 9
1.2.4 System Calls vs. Library Calls . . . 9
1.2.5 Kernel Mode and User Mode . . . 10
1.3 OS Abstractions and Course Structure . . . 11
1.3.1 Processes . . . 11
1.3.2 Files . . . 12
1.3.3 Address Spaces . . . 12
1.3.4 Security and Protection . . . 13

2 Process Management **15**

2.1 From Program to Process . . . 16
2.1.1 A Program Is Not a Process . . . 16
2.1.2 Life of a C Program . . . 16
2.1.3 Static and Dynamic Linking . . . 18
2.2 Process Creation and Execution . . . 19
2.2.1 Process Identification . . . 19
2.2.2 Process Creation: `fork()` . . . 19
2.2.3 Process Execution: the `exec*()` Family . . . 21
2.2.4 Running a Program and Coming Back . . . 22
2.2.5 Reading Man Pages . . . 23
2.2.6 Are `fork()` and `exec()` Wasteful? . . . 23
2.3 Processes in the Kernel . . . 24
2.3.1 Kernel-Space and User-Space Memory . . . 24
2.3.2 The Task List . . . 24
2.3.3 File Descriptors and Redirection . . . 25
2.3.4 Handling System Calls . . . 25
2.3.5 Process Time . . . 26
2.4 Process-Management System Calls in the Kernel . . . 28
2.4.1 `fork()` . . . 28

2.4.2 `execve()` . . . 29
2.4.3 `exit()` . . . 30
2.4.4 `wait()` and `waitpid()` . . . 30
2.4.5 Zombies . . . 31
2.5 Signals . . . 32
2.5.1 Signals Are Software Interrupts . . . 32
2.5.2 How Signals Are Generated . . . 32
2.5.3 The `kill` Command and Job Control . . . 32
2.5.4 The `kill()` System Call . . . 34
2.5.5 Standard Signals . . . 34
2.5.6 Changing Signal Handlers . . . 34
2.5.7 Waiting for a Signal . . . 35
2.5.8 Timers . . . 35
2.6 Process Organization . . . 36
2.6.1 Booting the Computer . . . 36
2.6.2 The First Process . . . 37
2.6.3 Orphans . . . 37
2.6.4 Observations . . . 39
2.7 Process Scheduling . . . 40
2.7.1 What Scheduling Is . . . 40
2.7.2 Process States . . . 40
2.7.3 Context Switching . . . 41
2.7.4 CPU-Bound and I/O-Bound Processes . . . 41
2.7.5 When to Schedule . . . 42
2.7.6 What Makes a Good Scheduler . . . 42
2.7.7 Scheduling Algorithms and Metrics . . . 43
2.7.8 First-Come, First-Served . . . 43
2.7.9 Shortest Job First . . . 44
2.7.10 Round-Robin . . . 45
2.7.11 Priority Scheduling . . . 46
2.7.12 Multilevel Feedback Queue . . . 47
2.8 Interprocess Communication . . . 49
2.8.1 What IPC Is and Why We Need It . . . 49
2.8.2 Two Ways to Do IPC . . . 49
2.8.3 Case Study: Pipes . . . 50
2.8.4 Shared Memory . . . 51
2.8.5 Race Conditions . . . 52
2.9 Critical Sections and Mutual Exclusion . . . 54
2.9.1 Mutual Exclusion and Critical Sections . . . 54
2.9.2 What a Good Solution Must Satisfy . . . 55
2.9.3 Attempt 1: Disabling Interrupts . . . 55
2.9.4 Attempt 2: A Lock Variable . . . 56
2.9.5 Attempt 3: Strict Alternation . . . 56
2.9.6 Attempt 4: Peterson's Algorithm . . . 57
2.9.7 Attempt 5: Spinlocks . . . 58
2.9.8 Attempt 6: Semaphores . . . 60
2.10 Classical IPC Problems . . . 62
2.10.1 The Producer-Consumer Problem . . . 62

2.10.2 The Dining Philosophers Problem . . . 64
2.10.3 The Readers and Writers Problem . . . 68
2.10.4 The Sleeping Barber Problem . . . 70
2.10.5 Summary . . . 71
2.11 Threads . . . 72
2.11.1 What a Thread Owns . . . 72
2.11.2 Why Threads? . . . 72
2.11.3 Thread Models . . . 73
2.11.4 POSIX Threads . . . 74
2.11.5 Synchronization in pthreads . . . 75
2.11.6 Thread Pools . . . 77
2.11.7 Thread Safety . . . 77
2.11.8 Summary . . . 78
2.12 Deadlocks . . . 79
2.12.1 Resource Allocation Graphs . . . 79
2.12.2 The Ostrich Algorithm . . . 80
2.12.3 Prevention: Attacking the Four Conditions . . . 80
2.12.4 Detection and Recovery . . . 83
2.12.5 Avoidance . . . 84
2.12.6 Summary . . . 84
2.12.7 Process Management in Retrospect . . . 84

A System Call Reference **86**

Chapter 1

Introduction

Supplementary reading:

- [2] Sections 1.1, 1.3, 1.5, and 1.6.
- [3] Chapters 1 and 2.
- [4] Chapter 2.
- [1] Sections 1.7 and 8.1.

This chapter follows the slide deck *1 — Introduction*. It answers what an operating system is, shows the ways programs and people interact with it, and previews the abstractions the rest of the course is organized around:

- Section 1.1: where the OS sits between users and hardware, and what happens when you type `ls`;
- Section 1.2: interrupts, system calls versus library calls, and kernel mode versus user mode;
- Section 1.3: the four OS abstractions—processes, files, address spaces, and protection—that structure the course.

1.1 What Is an Operating System?

1.1.1 Where the OS Fits In

Between a user and the computer hardware sits the operating system. Its job is to turn ugly hardware into beautiful abstractions. The OS hides the mechanics of the hard disk from you, for example, and lets you manipulate *files* instead.

Users do not deal with the OS directly, though. They deal with *processes*, the running images of programs. You have only one copy of Google Chrome, but you can open several browser windows from the same program file. Everything you run—accessing files, browsing the web, watching videos, playing games, doing the OS labs—runs as a process, which is why much of this course is about the process lifecycle, process management, and related issues.

1.1.2 Example: What Happens When You Type `ls`

When the user enters `ls`, which process runs, how does the OS react, and how does it work with the hardware to get the result back? Figure 1.1 traces the five steps.

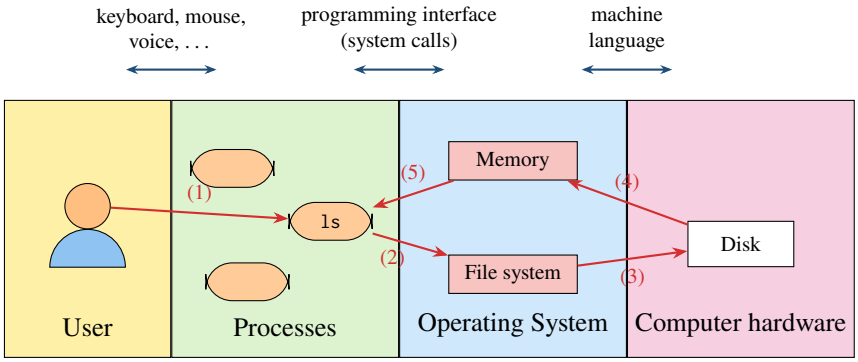


Figure 1.1: The OS between processes and hardware, and the five steps of running `ls`; the arrows on top are the three interfaces between neighboring layers (after slides 9–15).

1. Most commands typed in the shell start a new process, here one running `ls`.
2. `ls` asks the OS to read the directory. The OS contains the code needed to work with the file system; such code in the OS is called the *kernel*.
3. The file system module knows how to work with devices through *device drivers*, and issues I/O requests to the disk.
4. The OS allocates memory for the results.
5. The memory management subsystem copies the results into the memory of the `ls` process, which prints them.

Definition 1 (Kernel)

The *kernel* is the core of the operating system: the code, together with its data, that manages the hardware and provides services such as file systems, memory management, and process management to processes.

1.1.3 Two Viewpoints

Definition 2 (Operating System)

An *operating system* is a piece of software seen from two viewpoints.

- *Top-down*: the OS is an *extended machine*. It provides a clean abstraction through which processes access the machine.
- *Bottom-up*: the OS is a *resource manager*. It manages all parts of the system efficiently.

The rest of the course discusses the details of OS design from both angles.

1.2 Interacting with the OS

1.2.1 Three Points of View

Who interacts with the OS, and how, depends on who you are.

- *An ordinary user* sees the OS as something that runs programs, through the keyboard, the mouse, voice recognition, and so on. They need not even know what a process is—unless Google Chrome freezes and must be killed.
- *A software engineer* sees the OS as software that lets a program interact with the user, the hardware, or the OS itself, through *system calls*.
- *A hardware engineer* sees the OS as software that hosts *device drivers*. How does the computer handle input and output? What happens when the user types a key?

1.2.2 Interrupts

Answering the last question requires a little computer architecture. The CPU is executing some program code, and its *program counter* register points to the next instruction to be executed. Suddenly someone types a key.

Definition 3 (Interrupt)

A *hardware interrupt* is a signal from a device asking the CPU to interrupt the currently executing code (when permitted) so that the device's event can be processed in a timely manner. Each kind of interrupt has a unique *interrupt number*, and the *interrupt vector* is an array of addresses, indexed by interrupt number, pointing to the *interrupt handlers*.

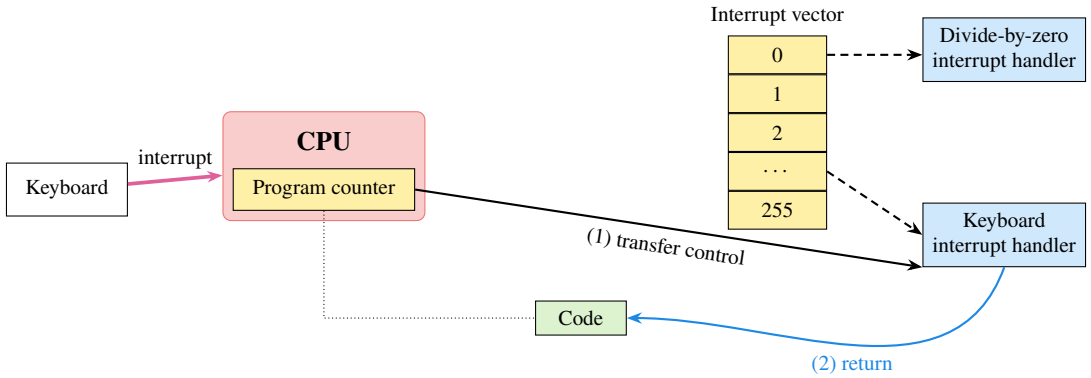


Figure 1.2: Handling a keyboard interrupt (after slides 19–23). The CPU looks up the handler in the interrupt vector, transfers control to it, and the handler returns to the interrupted code.

As Figure 1.2 shows, the CPU transfers control to the handler for that interrupt number, the handler processes the interrupt, and finally it returns control to the original code. Seen from the hardware side, then, the programs on a computer are just processes running on top of the OS, and they interact with it through system calls.

1.2.3 System Calls

Definition 4 (System Call)

A *system call* (syscall) looks like an ordinary function call, with one important distinction: its implementation is inside the OS kernel. Invoking a system call *traps* into the kernel. System calls are the programming interface between processes and the kernel.

Example 5 (`time()`)**.**

```
int add(int a, int b) {
  return a + b;
}
int main() {
  int s = add(1, 2);         // ordinary function call
  time_t t = time(NULL);     // system call: traps into the kernel
}
```

A process is running `main`. When it calls `time()`, it asks the kernel to run the `time()` system call; the kernel asks the hardware clock for the current time and returns the result. Note that the kernel is *not* an executing entity of its own: it is a body of compiled code and allocated memory, run by the process that trapped into it.

System calls are usually primitive and fundamental; `time()` just tells you what time it is. An OS provides system calls for process control, file management, device management, information maintenance, communications, and protection. *POSIX* (the Portable Operating System Interface) standardizes a common set. To check whether a function is a system call, consult `man syscalls`, the list of Linux system calls (section 2 of the manual).

1.2.4 System Calls vs. Library Calls

Which of `abs()`, `fopen()`, `fread()`, `malloc()`, `memcpy()`, and `printf()` are system calls? None of them.

Definition 6 (Library Function)

A *library function* is an ordinary function implemented in a library, not in the kernel. Library functions are compiled and packed into library files: `*.so` (shared object) on Linux, `*.dylib` (dynamic library) on macOS, and `*.dll` (dynamic-link library) on Windows.

A library function may itself invoke system calls. `fopen()`, implemented in the standard C library (e.g., `/lib/libc-2.17.so`), invokes the system call `open()`. Why reinvent the wheel? Because system calls are often too primitive and not programmer-friendly. Compare

```
open("file.txt", O_CREAT|O_WRONLY|O_TRUNC, S_IRUSR|S_IWUSR);
fopen("file.txt", "w");
```

1.2.5 Kernel Mode and User Mode

Definition 7 (Kernel Mode and User Mode)

The CPU has (at least) two separate modes of operation. The OS runs in *kernel mode* (a.k.a. privileged mode), with complete access to all the hardware and the ability to execute any instruction. All other programs run in *user mode*: only a subset of the machine instructions is available, and they cannot affect control of the machine or do I/O.

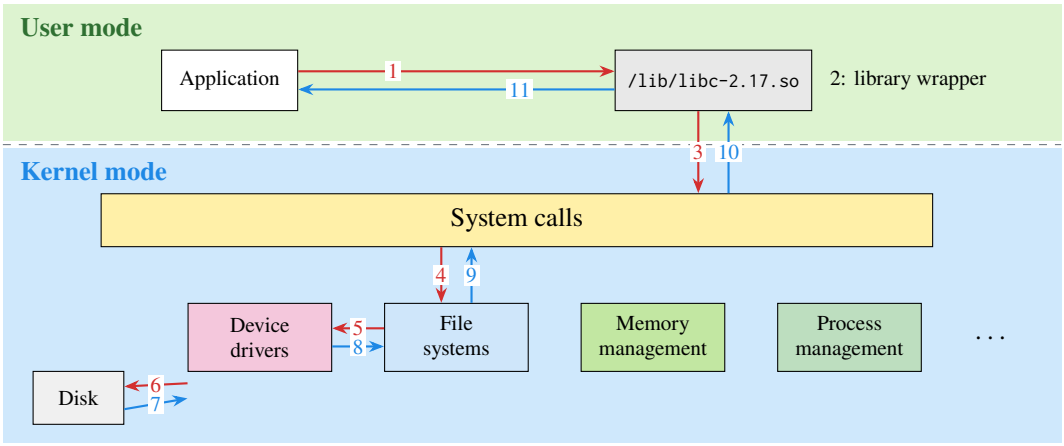


Figure 1.3: Opening a file with `fopen()` (after slides 35–36). Red: the request travels down; blue: the result travels back up.

Example 8 (`fopen()` Across the Mode Boundary)**.** Figure 1.3 numbers the steps.

1. The application calls `fopen()` in user mode.
2. The C library's wrapper prepares the request.
3. It calls `open()`, which TRAPs into the OS kernel.
4. The kernel allocates kernel data structures and hands the request to the file system.
5. The file system asks the device driver to read the file's metadata.
6. The driver issues a low-level disk read command.
7. The data is stored in memory.
8. The driver returns the file's metadata to the file system.
9. The file system finishes the open.
10. The system call returns the low-level *file descriptor* with RETURN-FROM-TRAP, back in user mode.
11. The library returns the high-level *file pointer* (`FILE *`) to the application.

Section 2.3 looks inside steps 3 and 10: how the kernel finds the right handler for a system call, and what a file descriptor is.

1.3 OS Abstractions and Course Structure

The OS provides four abstractions, and the course is organized around them:

Abstraction	Parts of the course
processes	Part 2: process management; Part 3: process synchronization
address spaces	Part 4: memory management
files	Part 5: storage management; Part 6: file system
security and protection	Part 7: security and protection

1.3.1 Processes

Consider the two commands `ls` and `ls /`. Both use the same program file, `/bin/ls`. They differ in their program arguments and in the internal state of their processes, such as running time.

Definition 9 (Process)

A *process* is an execution instance of a program. Multiple processes can execute the same program code, and a process is not bound to execute just one program. A process is an *active* entity: it maintains local state about its execution—which CPU core it runs on, which instruction is next, . . . —and that state changes over time.

Section 2.1 returns to the distinction between program and process from the build side.

Definition 10 (Shell)

The *shell*, the Unix command interpreter, is not part of the OS but an important program shipped with it. A shell starts when you log in, and it creates processes: when you enter a command, the shell, as the *parent* process, creates a *child* process that executes the command.

The shell itself has a parent, and children may have children of their own, so all processes form a *process tree* (Section 2.6). On a CIMS compute server:

```
$ pstree
systemd-+-sshd---bash---pstree
        ...
```

systemd is the ancestor of all processes, `sshd` is the SSH server, and `bash` is the shell. Inside a Docker container the tree looks different, since the container is a virtualized environment. The `ps` command displays information about processes, including each one's *PID* (process ID), a unique identification number:

```
$ ps
  PID TTY          TIME CMD
 2250 pts/23   00:00:00 bash
 2436 pts/23   00:00:00 ps
$ ps aux
USER   PID %CPU %MEM    VSZ  RSS TTY  STAT START    TIME COMMAND
root     1  0.0  0.0 194812 8008 ?    Ss   Jan14  121:09 /usr/lib/systemd/systemd
root     2  0.0  0.0      0    0 ?    S    Jan14    0:27 [kthreadd]
```

...

On processes, the course covers process-related functions and system calls (how to write a shell yourself, Lab 2), the process lifecycle and scheduling (how processes are created and how their death is handled), process control (how to suspend a process, what Ctrl-C does and how to change it), and process synchronization and deadlocks (how processes communicate and cooperate, Lab 3). Chapter 2 begins this part.

1.3.2 Files

Definition 11 (File and File System)

A *file* is a logical unit of information created by a process; think of it as an array of bytes, each of which you can read or write. A *file system* (FS) is how the OS uses a storage device to manage files: it maintains files, directories, allocated space, and free space.

Examples include APFS, ext4, FAT32, and NTFS. A file system is independent of the OS, and one OS can support several. File systems can be studied from two viewpoints: the *static* view asks how files are stored on the disk, and the *dynamic* view asks how the operations that update files are supported.

Example 12 (Fundamental File Operations)**.** Of `create`, `open`, `read`, `write`, `close`, `copy`, `delete`, `rename`, and `move`, not all are fundamental. `create` is a special case of `open`; `copy` can be implemented with `open`, `read`, and `write`; and `move` within the same file system is just `rename`.

On files, the course covers file-related functions and system calls, file system implementation (how well-known file systems work, why a file system becomes slow, how to recover a deleted file and why that does not always work, Lab 4, why a file can be lost on a physically healthy disk), and disks (what is inside a hard disk drive, and how multiple disks combine for performance, reliability, or both).

1.3.3 Address Spaces

When you run a program, it is loaded from the hard disk into main memory, and during execution its instructions are fetched from main memory into the CPU (Figure 1.4). There may be a hundred processes running "at the same time" with only a few CPU cores; Section 2.7 explains how.

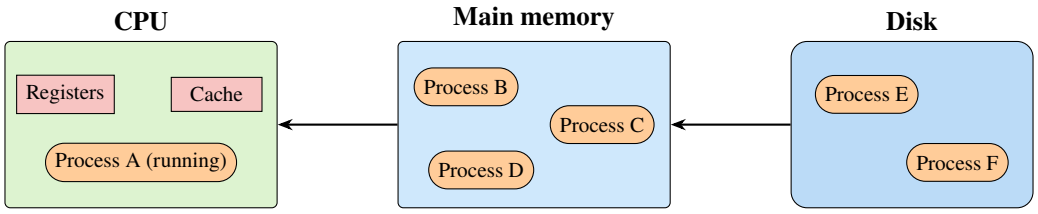


Figure 1.4: The memory hierarchy: programs move from disk to main memory to the CPU, and only a few processes run at any instant (after slide 52).

Definition 13 (Address Space)

The *address space* of a process is the range of (virtual) memory addresses it can use. A loader lays out the program in it as *segments*: text (program code), initialized data, uninitialized data, the heap, mapped files, and the stack. This organization is called *segmentation*.

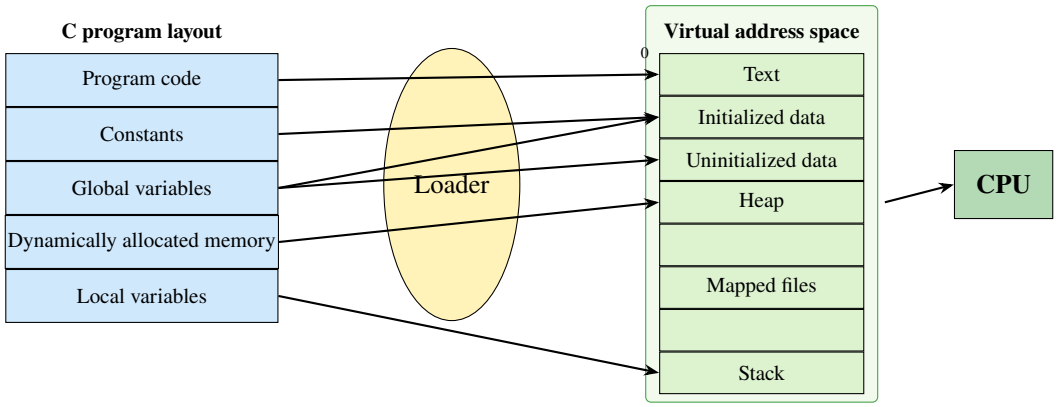


Figure 1.5: A loader maps a C program into the segments of a virtual address space (after slide 53).

Figure 1.5 shows which part of a C program goes where. Java seems not to have this layout, but in reality a Java program runs inside the Java virtual machine, which is itself just a process on the OS: `new String("hello")` creates an object that is a piece of dynamically allocated memory in the JVM process, obtained with `malloc()` and later `free()`-ed.

Why C? Real operating systems are virtually always written in C. C was invented for developing Unix; it is powerful, efficient, and predictable. Linus Torvalds puts it this way: "If you think like a computer, writing C actually makes sense." The flip side is that you must manage memory yourself: use `malloc()` and `free()` properly, initialize memory before using it, and be careful about bounds (Lab 1)—or meet `Segmentation fault`.

On address spaces, the course covers memory-related functions and system calls (why a program segfaults, whether a program that does not segfault manages memory correctly, how to write `malloc()` with system calls) and virtual memory (how processes can use more memory than the machine has, why a computer sometimes feels slow, how processes share memory).

1.3.4 Security and Protection

Definition 14 (Protection)

Protection is the OS's control over the access of processes or users to the resources defined by the system. The OS must provide means to specify the controls to be imposed and to enforce them.

Familiar attacks include viruses, worms, denial-of-service attacks, identity theft, and theft of service. The course covers protection-related functions and system calls (how the OS controls access, what file permissions mean), security best practices (how to store and verify passwords), and hacking (how root privilege is gained, legally and illegally, and how to protect yourself).

Key Point 15 (Summary)

An operating system is an extended machine that hides hardware behind clean abstractions, and a resource manager that shares the hardware among processes. Processes reach it through system calls, often wrapped by library functions, and the CPU's kernel and user modes keep them from bypassing it. Its four abstractions—processes, address spaces, files, and protection—structure the rest of the course.

Chapter 2

Process Management

Supplementary reading:

- [1] Chapter 7, Sections 8.1–8.5, 10.8, 10.9, 12.3–12.5, and 12.7.
- [4] Chapters 4–8, 26–28, and 30–32.
- [2] Sections 2.1–2.5 and 10.3, and Chapter 6.
- [3] Sections 2.9 and 3.3–3.7, and Chapters 4–8.

This chapter follows the slide deck *2 — Process Management*. It starts from the user's view of a process and works down into the kernel:

- Section 2.1: how a C program is built, and what distinguishes a process from a program;
- Section 2.2: creating processes with `fork()`, running programs with `exec*()`, and waiting with `wait()`;
- Section 2.3: the `task_struct`, file descriptors, and how the kernel handles a system call;
- Section 2.4: what `fork()`, `execve()`, `exit()`, and `wait()` change inside the kernel, and why zombies exist;
- Section 2.5: signals, job control, and signal handlers;
- Section 2.6: booting, the first process `init`, orphans, and the process tree;
- Section 2.7: process states, context switching, and scheduling algorithms from FCFS to MLFQ;
- Section 2.8: interprocess communication through pipes and shared memory, and race conditions;
- Section 2.9: critical sections, and six attempts at mutual exclusion, ending with semaphores;
- Section 2.10: the producer-consumer, dining philosophers, readers and writers, and sleeping barber problems;
- Section 2.11: threads, thread models, POSIX threads, mutexes and condition variables, and thread safety;
- Section 2.12: the four conditions for deadlock, and how to ignore, prevent, detect, or avoid deadlocks.

2.1 From Program to Process

2.1.1 A Program Is Not a Process

"What is a program?" is less obvious than it looks. A program is a piece of code—but *which* code? Any of the following layers can be called "the program":

- high-level languages: C, C++, Java, . . . ;
- intermediate languages: Java bytecode, LLVM IR, .NET CIL, . . . ;
- low-level languages: assembly;
- machine code.

A process is something else, and separating the two is the starting point of this unit: a program is static text, while a process is a dynamic, stateful entity.

Definition 16 (Process)

A *process* is an abstraction of a running program. It owns

- all the memory allocated to it,
- all the files it has opened, and
- accounting information: owner, running time, memory usage, . . .

So a process is not just "code that is running"; it is the full context the kernel uses to manage that running instance. In Linux that context is a structure called `task_struct` (Section 2.3). The process is the most central concept in an operating system. Before studying it, we follow a C source file all the way to something that can be executed.

2.1.2 Life of a C Program

The build pipeline has four stages, each with its own tool (Figure 2.1). `gcc hello.c -o hello` runs all four at once, but each can be run separately.

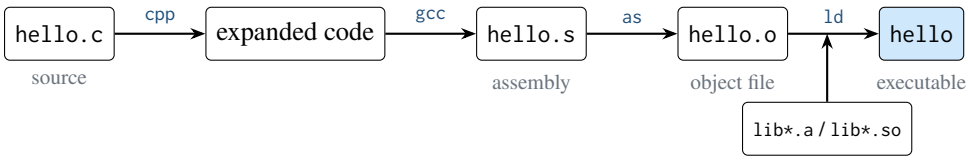


Figure 2.1: The build pipeline: preprocessor, compiler, assembler, linker (after slide 3).

The Preprocessor. The preprocessor (`cpp`) expands *directives* such as `#include`, `#define`, and `#ifdef`. It performs pure text substitution and knows nothing about C semantics. `gcc -E main.c` runs only this stage: `#include "header.h"` is replaced by the whole header, and a macro call like `MAX(1, 2)` is expanded in place to `((1) > (2) ? (1) : (2))`. Because expansion is textual, macro parameters must be parenthesized everywhere.

The Compiler. The compiler takes the expanded C code, checks the syntax, and emits assembly (`gcc`) or LLVM IR (`clang`). It also optimizes the code, sometimes dramatically.

Example 17 (Optimization Can Erase a Function)**.** Consider

```c
int main() {
  int x = 1;
  x = x - 1;
  return x;
}
```

`gcc -S`

```
pushq   %rbp
movq    %rsp, %rbp
movl    $1, -4(%rbp)
subl    $1, -4(%rbp)
movl    -4(%rbp), %eax
popq    %rbp
ret
```

`gcc -S -O2`

```
xorl    %eax, %eax
ret
```

Without optimization, the compiler faithfully pushes, stores, subtracts, reloads, pops, and returns. With `-O2` it sees that the function always returns 0 and emits two instructions.

Key Point 18

The code you write and the instructions the CPU executes need not correspond. Shifted line numbers and "optimized out" variables when debugging optimized code both come from this.

The Assembler. The assembler (`as`) converts assembly into an *object file*. The object file already contains machine code, but it is *not yet executable*: it still has unresolved symbols. Where `printf` lives, for instance, is not known yet.

The Linker. The linker (`ld`) combines object files and libraries into the final executable.

Definition 19 (Library)

A *library* is a collection of functions and variables. It comes in two forms:

	a.k.a.	Created with	Extension
static library	archive	`ar -rcs`	`.a`
dynamic library	shared object	`gcc -shared`	`.so` / `.dylib` / `.dll`

The three dynamic-library extensions are those of Linux, macOS, and Windows.

2.1.3 Static and Dynamic Linking

The two kinds of library correspond to two ways of linking, which differ in *where the library code ends up* (Figure 2.2).

- *Static linking.* The final program embeds every library function it uses. It is self-contained and needs no external files at run time; the cost is a larger binary, and a library upgrade requires relinking.
- *Dynamic linking.* The final program only records "I need library X". The OS maps the library in when the program runs. Many programs share one copy, saving memory and disk, and a library bug fix benefits all of them at once.

`ldd /bin/python3` lists the dynamic libraries a program depends on.

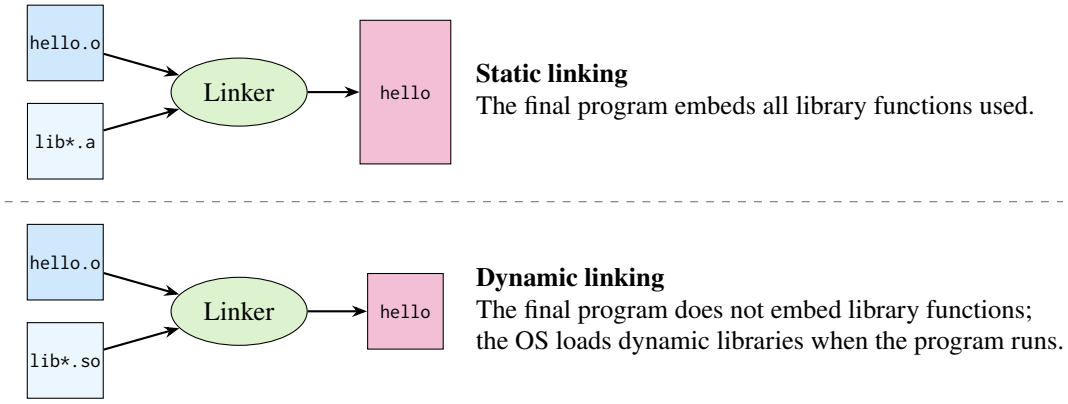


Figure 2.2: Static vs. dynamic linking (after slide 11).

The rest of the unit first studies the system calls a programmer uses to manage processes (create, execute, wait, signal) and then turns to the kernel to see what each one does inside.

2.2 Process Creation and Execution

2.2.1 Process Identification

Definition 20 (PID)

The OS gives each process a unique identifier, the *process ID* (PID). `getpid()` returns the PID of the caller, and `getppid()` returns the PID of its *parent*.

Running the same program repeatedly shows a different PID each time, while the parent PID stays the same: the parent is the shell that launched it.

2.2.2 Process Creation: `fork()`

Unix does not create a process by starting a new program from scratch. It *clones the current process*.

Definition 21 (`fork()`)

`fork()` creates a new *child* process that is a copy of the calling *parent*. Both continue executing from the point where `fork()` returns. It returns the child's PID (or -1 on failure) in the parent and 0 in the child.

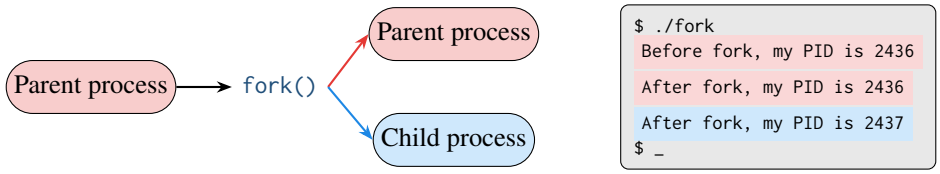


Figure 2.3: `fork()` clones the caller; both copies continue after the call (after slide 15).

Example 22 (`fork.c`)**.**

```c
int main() {
  printf("Before fork, my PID is %d\n", getpid());
  fork();
  printf("After fork, my PID is %d\n", getpid());
}
```

"Before" is printed once and "After" twice, with different PIDs (Figure 2.3). The child does not run from the beginning; it starts when `fork()` returns. Parent and child execute the same code, so every line after `fork()` runs in both, while "Before" is printed only once because the child did not exist yet.

Telling Parent from Child. Since both processes run the same code, the return value is the only way to tell them apart:

```c
int ret = fork();
if (ret == 0) {
    // child
} else if (ret > 0) {
```

```
    // parent; ret is the child's PID
} else {
    // fork failed
}
```

The design is deliberate. The child can always find its parent with `getppid()`, but the parent has no other way to learn who its new child is, so the useful return value goes to the parent.

What the Child Inherits. The child inherits, but is independent from, the parent's

- program code,
- program counter (so both continue at the same point),
- memory (global and local variables, dynamically allocated memory), and
- open files.

It receives a *copy*; afterwards each side modifies its own. The child differs in the following:

Item	In the child
return value of `fork()`	0 (the parent gets the child's PID, or -1)
PID	a new PID, *not necessarily* the parent's PID $+1$
parent	the calling process, not the grandparent
running time	reset to 0
file locks	not inherited

Buffering and `fork()`. Example 23 (A Classic Trap)**.**

```
int main() {
  printf("Hello ");
  fork();
  printf("2250\n");
}
```

One might expect `Hello 2250` followed by `2250`. The output is `Hello 2250` *twice*.

`printf()` is a library function built on the `write()` system call, and the `FILE` structure holds a *buffer* to reduce the number of system calls. `"Hello "` has no newline, so at the time of `fork()` it is still in the buffer. The child inherits the buffer together with its contents, and both processes eventually flush it. The C standard library uses three buffering policies:

Policy	Behavior	Default for
unbuffered	`write()` immediately	`stderr`
line-buffered	`write()` at a newline	`stdin`, `stdout`
fully buffered	`write()` when full or at exit	files

The fix is to flush before forking, `fflush(stdout)`, or to change the policy with `setvbuf()`.

Remark 24. This also explains why output on `stderr` survives a crash while output on `stdout` may be lost: it may still be sitting in the buffer.

Who Runs First. On a single processor only one process runs at a time. Whether the parent or the child runs first after `fork()` is decided by the OS scheduler and is *unknown* to the program. Code that assumes either order is wrong.

2.2.3 Process Execution: the `exec*()` Family

`fork()` can only duplicate the caller. To run a *different* program we need `execve()`.

Definition 25 (`execve()`)

```
int execve(const char *filename, char *const argv[], char *const envp[]);
```

`execve()` replaces the current process image with a new program. On success it does not return.

Example 26 (Code After `exec` Is Dead)**.**

```
int main() {
  printf("Before exec\n");
  execl("/bin/ls", "/bin/ls", NULL);
  printf("After exec\n");     // never reached
}
```

Once `execl` succeeds, the process runs `ls` and there is no way back. The last line runs only if the call *fails*, so a call to `exec` is normally followed by error handling.

The process discards its memory (global, local, and dynamically allocated) and its registers, including the program counter. It keeps its PID, its relationships with other processes, its running time, and so on.

Key Point 27

`fork()` creates a process; `exec()` changes the program a process runs. The "container" stays the same while its contents change.

Naming Convention. Calling `execve()` directly is verbose, so there is a family of wrappers whose suffixes follow a pattern:

Suffix	Meaning
(none)	the first argument is a *path name*, e.g. `/bin/ls`
`p`	the first argument is a *file name*, e.g. `ls`, searched in `PATH`
`l`	arguments as a *list*: `execl("/bin/ls", "/bin/ls", "-a", NULL)`
`v`	arguments as an *array*: `execv("/bin/ls", argv)`
`e`	an explicit *environment* array; without `e`, inherit the current one

The combinations are `execl`, `execlp`, `execle`, `execv`, `execvp`, and `execve`. The `l` variants are variadic and must end with `NULL`. By convention `argv[0]` is the program name itself, which is why the path usually appears twice.

Environment Variables. Environment variables are a set of strings maintained by the shell, and many programs read them. `main` can take a third parameter, organized exactly like `argv` as a `NULL`-terminated array of string pointers:

```
int main(int argc, char **argv, char **envp) {
  while (*envp)
    printf("%s\n", *envp++);
}
```

Common ones are `PATH` (where to search for commands), `HOME`, `SHELL`, `PWD`, and `LANG`.

2.2.4 Running a Program and Coming Back

The library function `system()` runs a command and then *returns to the original code*. It is implemented with `fork()` and `exec()`, running `/bin/sh -c command`.

Example 28 (`my_system`, First Attempt)**.**

```
int my_system(const char *command) {
  if (fork() == 0) {
    execl("/bin/sh", "/bin/sh", "-c", command, NULL);
    exit(-1);                  // only reached if execl fails
  }
  return 0;
}
```

This version works, but the output order is nondeterministic: the parent continues without waiting for the child, so "After system" may appear before the output of the command. The `exit(-1)` is essential: if `execl` fails and the child does not exit, it continues into the caller's code and "After system" is printed twice.

What we want is not to "let the child run first"—the scheduler is not ours to control—but to *suspend* the parent and *wake it up* once the child has finished.

Definition 29 (`wait()`)

`wait()` suspends the calling process until one of its children terminates. A non-`NULL` argument receives the child's status, such as its exit code; `NULL` means "don't care". If the caller has no children, `wait()` returns -1 immediately. `waitpid()` is the general version: it can wait for a *specific* child, or for children that are stopped or resumed.

Example 30 (`my_system` with `wait()`)**.**

```
int my_system(const char *command) {
  if (fork() == 0) {
    execl("/bin/sh", "/bin/sh", "-c", command, NULL);
    exit(-1);
```

```
    }
    wait(NULL);                  // block until a child terminates
    return 0;
}
```

2.2.5 Reading Man Pages

Man pages give the headers to include, compile flags, arguments, return values, and error conditions. They are divided into sections, and the section number matters:

Section	Contents
1	executable programs or shell commands
2	system calls (functions provided by the kernel)
3	library calls (functions within program libraries)

`man wait` is `man 1 wait`, the shell built-in; the system call is `man 2 wait`. `man man` lists all sections.

2.2.6 Are `fork()` and `exec()` Wasteful?

It seems wasteful that `fork()` copies all the parent's memory only for `exec()` to discard it. In reality nothing is copied up front.

Definition 31 (Copy-on-Write)

After `fork()`, the child shares memory with the parent *copy-on-write* (COW). As long as neither process modifies the shared memory, no copy is made. When one of them writes to a piece of it, the OS copies that piece, and the modification happens privately.

So the common pattern of `fork()` immediately followed by `exec()` copies almost nothing.

Key Point 32 (Summary)

A process is created by cloning: `fork()` copies the caller, and the clone inherits much, but not everything, from its parent. A process is the entity that runs a program, and it may run more than one program in its lifetime: the `exec*()` family replaces the program a process runs; it does not create a process. Where the very first process comes from is the subject of Section 2.6.

2.3 Processes in the Kernel

The previous sections looked at processes from the programmer's side. This section turns to the kernel: what *is* a process inside the kernel, and how is a system call carried out?

2.3.1 Kernel-Space and User-Space Memory

Definition 33 (User Space and Kernel Space)

System memory is divided into two parts.

	Contents	Accessible by
user space	program code, process memory, . . .	user code and kernel code
kernel space	kernel code and data structures, drivers, . . .	kernel code only

This boundary is the foundation of every protection mechanism in the OS. User programs cannot even address kernel space, while the kernel can access all memory—which is also why a kernel bug is far more serious than a user bug.

2.3.2 The Task List

Definition 34 (Process Control Block)

For each process the kernel keeps a *process control block* (PCB) containing everything related to the process: PID, running time, file descriptors, signal handlers, parent and children, . . . In Linux it is `struct task_struct`. All PCBs are linked into a doubly linked list, the *task list*, stored in kernel-space memory.

The abstract notion of a process is, concretely, this structure (Figure 2.4); `getpid()` merely reads one of its fields.

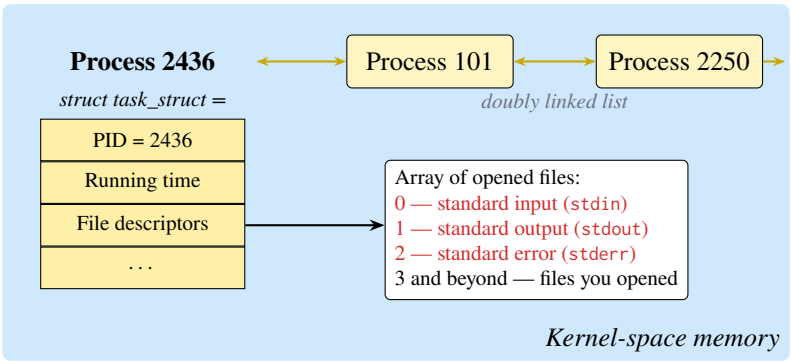


Figure 2.4: The task list. Each `task_struct` is a process control block, and its file descriptors index an array of open files (after slide 36).

2.3.3 File Descriptors and Redirection

Definition 35 (File Descriptor)

The `task_struct` holds an array of the process's open files. An index into this array is a *file descriptor* (fd): 0 is standard input (`stdin`), 1 is standard output (`stdout`), 2 is standard error (`stderr`), and files the program opens get descriptors 3 and beyond.

The first three are already set up by the shell when the program starts, which is why the first file you open gets descriptor 3. Since "standard output" is just entry 1 of the array, *redirecting* it means changing that entry.

Example 36 (Redirecting `stdout` with `dup2()`)**.**

```
int main() {
  printf("Hello 2250\n");                    // -> terminal

  int fd = open("output.txt",
                O_CREAT|O_WRONLY|O_TRUNC,
                S_IRUSR|S_IWUSR);
  dup2(fd, 1);    // duplicate the file descriptor onto fd 1
  close(fd);      // close the unused file descriptor

  printf("Hello 2250 again\n");              // -> output.txt
}
```

After `dup2(fd, 1)`, descriptor 1 refers to `output.txt`, and everything written to `stdout`—including by `printf`, which knows nothing about the change—goes to the file.

This is exactly how the shell implements `./prog > output.txt`: it calls `dup2()` in the child after `fork()` and before `exec()`. Because `exec()` preserves the file descriptor table, the new program starts already redirected.

2.3.4 Handling System Calls

A process switches its execution from *user mode* to *kernel mode* by invoking a system call, and switches back when the call returns. It is the same process changing execution mode, not a different process: the program counter still belongs to it, but now points into kernel code with access to kernel space.

Definition 37 (System Call)

Each system call has a unique *syscall number*. To make a system call, a process places the number in a designated CPU register (`%rax` on x86-64) and executes a *TRAP* instruction. The kernel starts executing at the *syscall dispatcher*, which looks up the number in the *syscall table* and invokes the corresponding *handler*. The handler finishes with a *RETURN-FROM-TRAP* instruction, switching back to user mode.

Example 38 (`getpid()`)**.** Follow Figure 2.5. The CPU is running the process in user mode. The process puts the syscall number of `getpid()`, 39, into the register and executes TRAP. The dispatcher finds entry 39 in the table and calls `sys_getpid()`, which reads the PID from the process's `task_struct` and executes RETURN-FROM-TRAP.

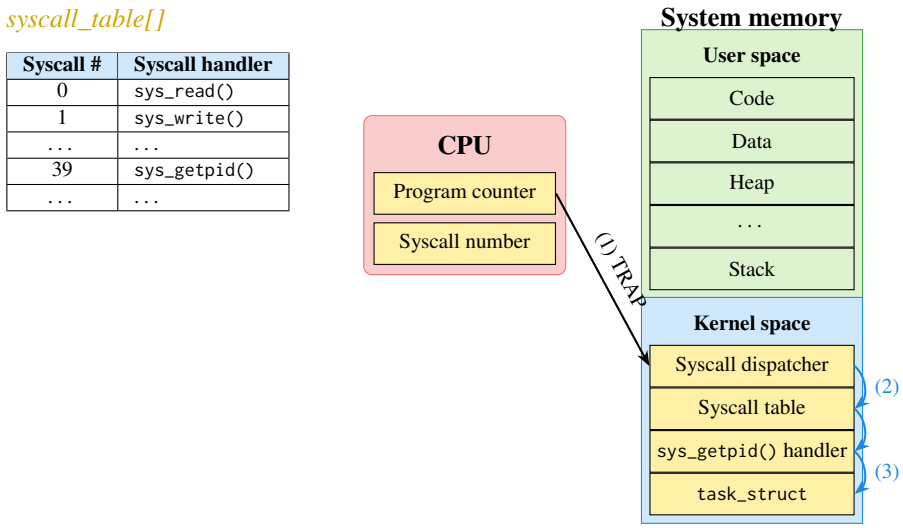


Figure 2.5: Handling `getpid()` (after slides 39–41): (1) trap into the dispatcher; (2) look up syscall number 39 in the syscall table; (3) `sys_getpid()` reads the PID from the `task_struct`, then returns from the trap.

Key Point 39

User code cannot jump to arbitrary kernel addresses. TRAP enters the kernel at a single place, the dispatcher, and the kernel decides which handler runs. The syscall table is the strictly limited list of entry points the kernel opens to user space.

2.3.5 Process Time

Since execution is divided between user mode and kernel mode, so is the running time of a process:

$$\text{running time} = \text{user time} + \text{system time}. \tag{2.1}$$

User time is spent executing the program's own code; *system time* is spent executing system calls in the kernel. Some system calls, I/O in particular, can take a long time.

Example 40 (The `time` Command)**.** The two programs below are equivalent, but the first makes ten times as many `write()` system calls (`stdout` is line-buffered):

```
// time1.c: 1,000,000 iterations, one newline each
for (int i = 0; i < LOOP_MAX; ++i)
    printf("\n");

// time2.c: 100,000 iterations, ten newlines each
for (int i = 0; i < LOOP_MAX / 10; ++i)
    printf("\n\n\n\n\n\n\n\n\n\n");
```

	`time ./time1`	`time ./time2`
`real` (wall clock)	10.045 s	6.808 s
`user`	0.204 s	0.090 s
`sys`	3.793 s	2.218 s

Two conclusions follow. First, system calls significantly affect performance: `sys` dwarfs `user`, so the time goes into entering the kernel rather than into the program's own logic. Second, `real` is much larger than `user` + `sys`; the difference is time the process spends *off* the CPU, descheduled or blocked on I/O. This is also the point of buffering: batching many small writes into one large write reduces the number of kernel entries.

2.4 Process-Management System Calls in the Kernel

Section 2.3 described how a system call is handled in general. This section takes `fork()`, `execve()`, `exit()`, and `wait()` one at a time and asks which kernel data structures each of them changes. At that level, phenomena such as zombie processes are no longer mysterious.

2.4.1 `fork()`

From the programmer's perspective, `fork()` is invoked once and returns in both the parent and the child. In the kernel it does three things.

1. *Duplicate the* `task_struct` and update the copy: allocate a new PID (2436 → 2437), reset the running time to 0, point the child's *Parent* field at the parent, and add the child to the parent's *list of children*.
2. *Duplicate the user-space memory*: text, initialized data, uninitialized data, heap, mapped files, and stack. This is done copy-on-write: both processes first share the same physical pages, marked read-only, and a write triggers a page fault in which the kernel copies just that page.
3. *Set two return values*: the child's PID (2437) in the parent's saved context and 0 in the child's.

The third step is how "one call returns twice" is implemented: two processes each return once, with different values (Figure 2.6).

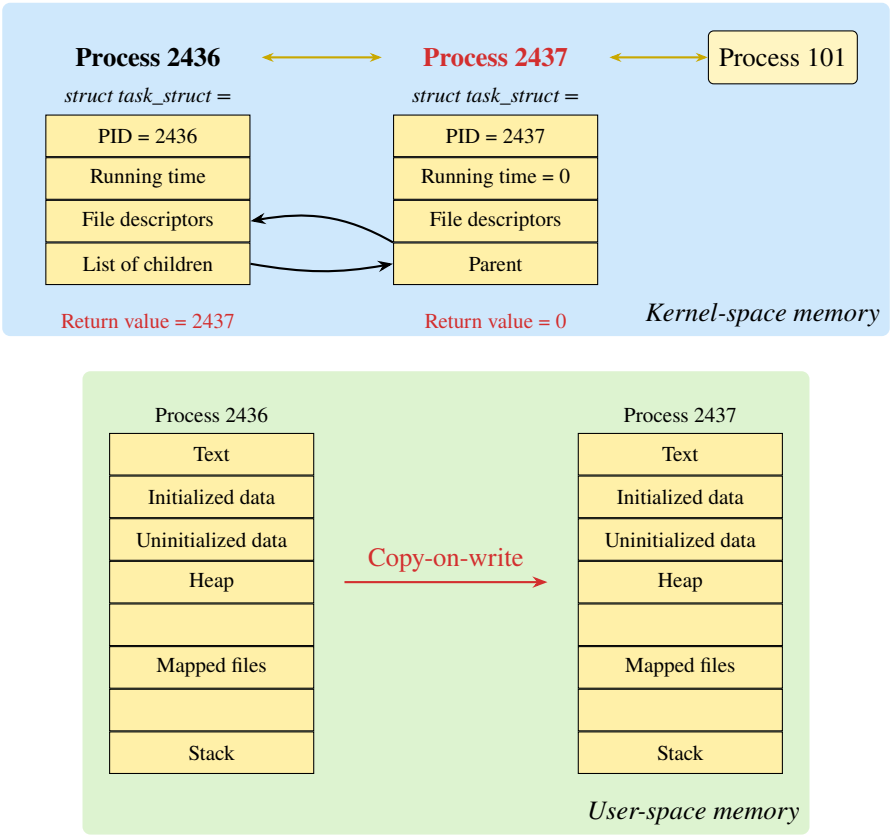


Figure 2.6: `fork()` in the kernel (after slides 49–50). Top: a new `task_struct` is linked into the task list, with Parent and children pointers and two different return values. Bottom: the address space is duplicated copy-on-write.

Example 41 (Parent and Child Writing the Same File)**.**

```
int main() {
  FILE *fp = fopen("file.txt", "w");
  pid_t pid = fork();
  for (int i = 0; i < 5; ++i) {
    fprintf(fp, "[%d] %d\n", getpid(), i);
    fflush(fp);
    sleep(rand() % 2);
  }
  fclose(fp);
  if (pid) wait(NULL);
}
```

The lines of the two processes *interleave* in `file.txt` (`[2436] 0`, `[2437] 0`, `[2436] 1`, ...) but never overwrite each other. Both descriptors point to the same kernel open-file entry, so they share the file offset: each write advances it for the other process too. The order of interleaving depends on scheduling.

2.4.2 `execve()`

From the programmer's perspective, `execve()` never returns (on success). In the kernel:

- the `task_struct` is mostly kept—PID, relationships, file descriptors, and running time remain;
- the user-space memory is reset, and the code of the new program is loaded into the text segment;
- the CPU registers, including the program counter, are reset to the entry point of the new program.

Since the program counter now points into the new program, the old code never runs again: this is the implementation-level reason why `execve()` does not return (Figure 2.7).

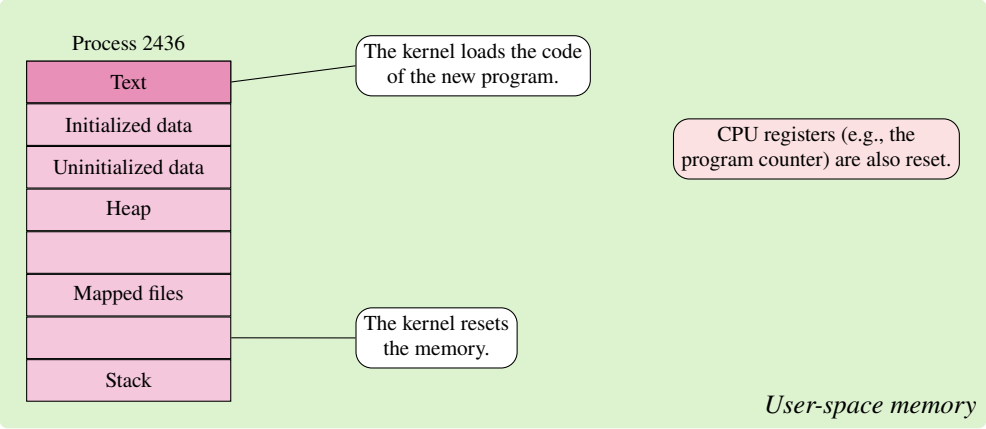


Figure 2.7: `execve()` keeps the process but rebuilds its user-space memory (after slide 55).

	`task_struct`	user-space memory
`fork()`	create a new one	duplicate (copy-on-write)
`execve()`	keep	reset and load the new program

2.4.3 `exit()`

To understand `wait()`, we first look at `exit()`, which is called automatically when a program reaches its end. The kernel

1. cleans up most of the kernel-space memory allocated to the process and all of its user-space memory;
2. closes all open files;
3. but keeps the PID and the `task_struct`;
4. sends the `SIGCHLD` signal to the parent, and `exit()` returns.

Definition 42 (Zombie)

A process that has exited but whose PID and `task_struct` have not yet been released is a *zombie*. It no longer runs and its storage is minimal, but it is still in the system.

Why can the kernel not remove the `task_struct` right away? Because the parent may still want to know *how* the child died: its exit code, or whether it was killed by a signal. That information lives in the `task_struct`, and the kernel cannot decide on the parent's behalf that it does not care.

Key Point 43

A zombie is a death certificate waiting to be collected by the parent.

2.4.4 `wait()` and `waitpid()`

Parent and child run concurrently, so the moment the parent calls `wait()` is independent of the moment the child exits. By default a process *ignores* `SIGCHLD` unless it calls `wait()` or `waitpid()`. Two cases arise (Figure 2.8).

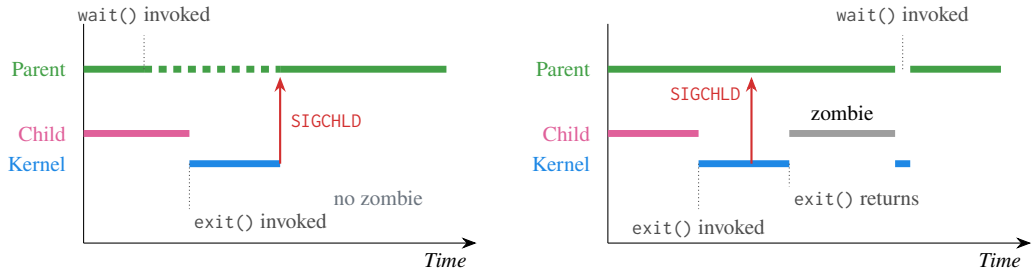


Figure 2.8: Left: `wait()` before `SIGCHLD` arrives (after slide 71). Right: `wait()` after `SIGCHLD` arrives; the zombie lives until the parent calls `wait()` (after slide 75).

Case 1: `wait()` Before `SIGCHLD` Arrives.

1. The kernel sets a `SIGCHLD` signal handler for the process, a function pointer stored in its `task_struct`, to run when the signal arrives.
2. The kernel puts the process in the *blocked* (a.k.a. sleeping) state, in which it does not use the CPU.

3. When `SIGCHLD` arrives, the parent's signal handler is invoked—not the original program code.
4. The handler removes the signal and destroys the zombie child, releasing its PID and `task_struct`.
5. The handler is removed, the process goes back to ignoring `SIGCHLD`, and it returns to user space where it left off.

From the user code's point of view, "`wait()` returned". In this case no zombie wanders around in the system: the child is reaped the instant it becomes one.

Case 2: `wait()` After `SIGCHLD` Arrives. The child has already exited, `SIGCHLD` has been delivered and is *pending*, and a zombie exists. When the parent finally calls `wait()` and sets the handler, it finds the signal already there, removes it, and reaps the zombie immediately, without blocking. In this case a zombie does exist, but only up to the moment the parent calls `wait()`.

2.4.5 Zombies

`wait()` and `waitpid()` exist to *reap* zombie children. Linux marks zombies as `<defunct>`, with state `Z`:

```
$ ps x
  PID TTY      STAT   TIME COMMAND
 2436 pts/0    SN+    0:00 ./zombie
 2437 pts/0    ZN+    0:00 [zombie] <defunct>
```

After the parent calls `wait()`, process 2437 disappears from the list.

Reaping is a matter of system resource management: each zombie holds a PID, and the PID space is finite (`cat /proc/sys/kernel/pid_max` prints, e.g., 65536).

Example 44 (Running Out of PIDs)**.**

```
int main() { while (fork()); }
```

The parent forks forever, and every child exits at once and becomes a zombie that nobody reaps. The PIDs are soon exhausted, after which the system cannot create any new process—not even `ls` or `poweroff`, since each of them must be forked too.

Key Point 45

Never leave zombies in the system.

Key Point 46 (Summary)

It is important to know the internal workings of system calls. Some of them have surprising results—`fork()` returns twice, `execve()` never returns, an exited process stays around—and understanding them clarifies process control in the OS. At the very least, you no longer have to guess what happens after calling a system call.

2.5 Signals

2.5.1 Signals Are Software Interrupts

Definition 47 (Signal)

A *signal* is a form of inter-process communication (IPC): a process can send a signal to another process. When a signal arrives, the OS interrupts the normal control flow of the target process and runs a *signal handler* instead.

Interrupting the normal flow to run other code and then coming back is exactly what a hardware interrupt does, so signals are also called *software interrupts*. The most familiar example is pressing Ctrl-C:

1. it sends the `SIGINT` signal to the current process;
2. the *default handler* of `SIGINT` terminates the process;
3. but the process may *supply its own handler* to override this behavior.

The third point matters: Ctrl-C is not a forced kill. It only delivers a signal, and the program may choose not to die.

2.5.2 How Signals Are Generated

From user space:

- the keyboard: Ctrl-C sends `SIGINT`, Ctrl-Z sends `SIGTSTP` (terminal stop), and Ctrl-\ sends `SIGQUIT`;
- commands such as `kill` and `top`;
- the `kill()` system call.

From the kernel or the hardware:

- `SIGCHLD` (a child stopped or terminated) comes from the kernel;
- `SIGFPE` (floating-point exception, e.g. division by zero) comes from the CPU;
- `SIGSEGV` (invalid memory reference) is sent by the CPU to the kernel, and by the kernel to the process.

Remark 48. The full chain of a segmentation fault: the CPU detects an illegal access during address translation, raises a hardware exception, the kernel takes over and turns it into a `SIGSEGV` for the offending process, and the default handler terminates the process and dumps core. A "segmentation fault" is, in the end, a signal.

2.5.3 The `kill` Command and Job Control

The `kill` command sends `SIGTERM` to the target process by default, and the default handler of `SIGTERM` terminates it:

```
$ ps -u $USER
  PID TTY          TIME CMD
```

```
 2250 pts/10   00:00:00 loop
$ kill 2250
```

The terminal running the process then shows `Terminated`. But `kill` can send *any* signal; the name is misleading, as it does not "kill", it "signals".

```
$ kill -TSTP 2250      # stopped: the shell prints [1]+ Stopped
$ kill -CONT 2250      # resumed: its output mixes with the shell prompt
```

After `SIGCONT`, the program's output interleaves with the shell prompt. Explaining why leads to job control.

Definition 49 (Foreground and Background Jobs)

A *foreground job* is a child that the shell is waiting for: the shell calls `fork()` and `exec()` and then blocks in `waitpid()`, so it does not respond to the user meanwhile. A *background job* is a child that runs concurrently with the shell while the shell is not waiting for it.

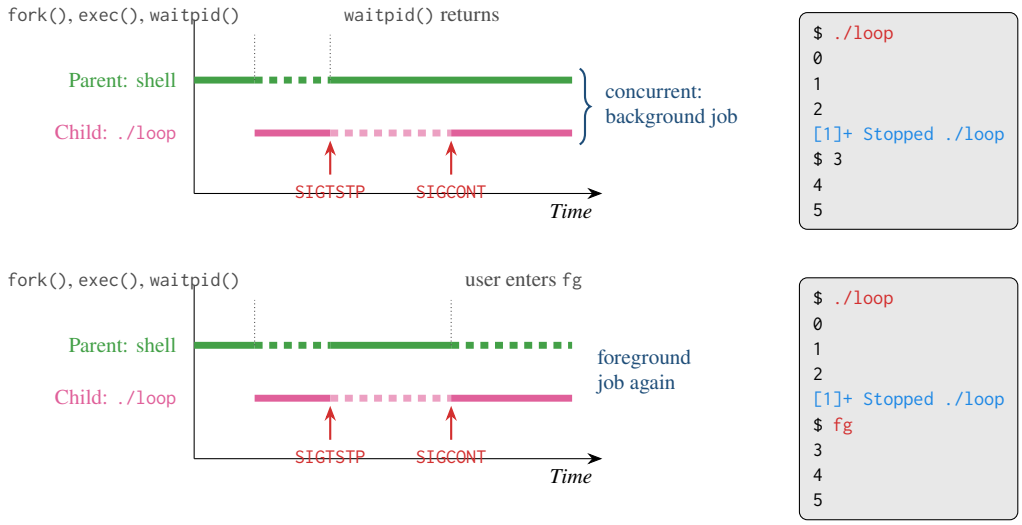


Figure 2.9: Top: after `SIGCONT` the shell is not waiting, so `./loop` runs as a background job (after slide 87). Bottom: `fg` sends `SIGCONT` and calls `waitpid()` again (after slide 88).

Figure 2.9 traces the experiment.

- While `./loop` runs in the foreground, the shell is blocked in `waitpid()`.
- `SIGTSTP` stops the child; `waitpid()` returns, and the shell regains control, printing `[1]+ Stopped` and a prompt.
- `SIGCONT` resumes the child, but the shell does *not* call `waitpid()` again. The two now run concurrently: the child has become a background job, and its output mixes with what you type.
- The `fg` command sends `SIGCONT` *and* blocks in `waitpid()` again, bringing the job back to the foreground. Implementing `fg` yourself therefore takes `kill(pid, SIGCONT)` followed by `waitpid(pid, ...)`.

2.5.4 The `kill()` System Call

Just like the command, the `kill()` system call sends a signal to a process, and the library function `raise()` sends a signal to the caller itself:

```
int kill(pid_t pid, int sig);
int raise(int sig);             // equivalent to kill(getpid(), sig)
```

2.5.5 Standard Signals

`man 7 signal` gives the full list. The common ones:

Signal	Description	Default handler
`SIGINT`	interrupt from keyboard (Ctrl-C)	terminate
`SIGTERM`	termination (default of `kill`)	terminate
`SIGKILL`	kill, no matter what	terminate (cannot be overridden)
`SIGTSTP`	stop typed at terminal (Ctrl-Z)	stop
`SIGSTOP`	stop, no matter what	stop (cannot be overridden)
`SIGCONT`	continue if stopped	continue
`SIGCHLD`	child stopped or terminated	ignore
`SIGFPE`	floating-point exception	terminate and dump core
`SIGSEGV`	invalid memory reference	terminate and dump core
`SIGALRM`	timer set by `alarm()`	terminate

Key Point 50

`SIGKILL` and `SIGSTOP` cannot be caught, blocked, or ignored. They are the administrator's last resort; otherwise a rogue program could intercept every signal and become impossible to stop. `kill -9` sends `SIGKILL`. Note also that `SIGCHLD` is ignored by default, which is the precondition for zombies (Section 2.4).

2.5.6 Changing Signal Handlers

The `signal()` system call changes a signal handler. Its prototype is one of the hardest declarations in C to read:

```
void ( *signal(int sig, void (*handler)(int)) ) (int);
```

A `typedef` untangles it:

```
typedef void (*sighandler_t)(int);
sighandler_t signal(int sig, sighandler_t handler);
```

The first argument is the signal number, and the second is a pointer to the new handler, a function taking an `int` and returning `void`. The return value is the *old* handler, so that it can be restored later.

Example 51 (Changing the Behavior of Ctrl-C)**.**

```c
void handler(int sig) {
  printf("...Ouch!\n");
}

int main() {
  signal(SIGINT, handler);
  for (int i = 0; ; ++i) {
    printf("%d\n", i);
    sleep(1);
  }
}
```

Ctrl-C now prints `...Ouch!` and the loop continues. To stop the program, use `kill -9` (`SIGKILL` cannot be overridden) or send `SIGTERM` from another terminal. The handler's parameter `sig` is the signal number, so one function can be registered for several signals and tell them apart.

2.5.7 Waiting for a Signal

The `pause()` system call puts the process to block (sleep) until either a signal that is *handled* by the process arrives, or a signal that *terminates* the process arrives.

Example 52 (`pause()`)**.**

```c
void handler(int sig) { }      // empty, but it must exist

int main() {
  signal(SIGINT, handler);
  printf("Press Ctrl-C to continue...\n");
  pause();
  printf("...Adiós!\n");
}
```

The empty handler cannot be omitted. Without it, `SIGINT` takes its default action and terminates the process, so the last line would never run: `pause()` returns only for a signal that is handled.

2.5.8 Timers

The `alarm()` system call sets up a one-time asynchronous timer for the process. When it expires, the process receives `SIGALRM`, whose default handler terminates the process; the handler can be overridden. The `setitimer()` system call sets up a *periodic* interval timer. Together these suggest how `sleep()` can be built: set `alarm(n)`, install an empty `SIGALRM` handler, and call `pause()`.

Key Point 53 (Summary)

A signal is a kind of software interrupt with many quirky behaviors. `kill()` is not meant to kill anyone; it sends signals. When in doubt, read the man page.

2.6 Process Organization

This section answers two questions: where does the *first* process come from, and what happens to a child whose parent dies first? Together the answers give the organization of processes in Linux: a single tree rooted at `init`.

2.6.1 Booting the Computer

At a high level, booting takes three steps:

1. the BIOS finds the boot device;
2. the boot code on the boot device is executed;
3. the boot code starts the operating system.

Definition 54 (BIOS)

The *BIOS* (Basic Input/Output System) is *firmware*, i.e., software providing low-level control of the hardware. It is stored in a ROM or flash chip on the motherboard, runs first when the machine is powered on, checks all attached devices, and finds the boot device. Newer systems replace it with *UEFI* (Unified Extensible Firmware Interface), a complete boot manager.

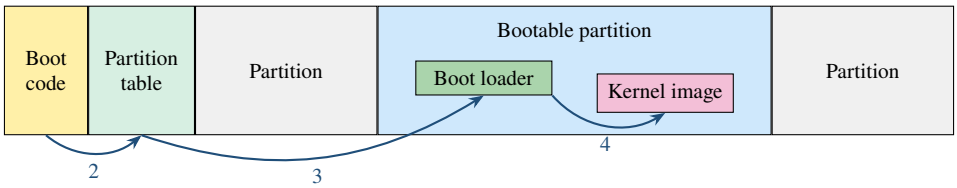


Figure 2.10: The boot device (after slide 99). The numbers refer to the steps in the text.

On the boot device (Figure 2.10):

1. the boot code in the boot device's first sector is executed;
2. the boot code reads the partition table and locates the bootable partition;
3. the boot loader in the bootable partition is executed;
4. the boot loader starts the OS kernel.

The boot loader (e.g., GRUB) locates and boots the kernel image, which is just a file:

Linux	`/boot/vmlinuz-*`
macOS	`/System/Library/Kernels/kernel`
Windows	`C:\Windows\System32\ntoskrnl.exe`

The kernel then initializes the memory layout, the device drivers, and so on, and creates the first process.

2.6.2 The First Process

Definition 55 (`init`)

During booting, the kernel creates the first process, `/sbin/init`, with PID 1. Newer Linux systems replace `init` with systemd; the difference can be ignored in this course.

`init` creates further processes with `fork()` and `execve()`, reading its configuration in `/etc/rc.d` to decide which programs to run. `pstree` shows the resulting process tree. `init` keeps running in the background until the system shuts down; the next section explains why it must.

2.6.3 Orphans

Definition 56 (Orphan and Reparenting)

A process whose parent has terminated is an *orphan*. `init` automatically adopts all orphan processes; this is called *reparenting*.

Consider the chain `init` → shell → program: if the shell terminates first, the program becomes an orphan and `init` adopts it. Without reparenting the orphan would have no parent and the process tree would break into a forest; reparenting keeps it a single tree.

Example 57 (Observing Reparenting)**.**

```c
int main() {
  if (fork() == 0) {
    // child process
    printf("My parent's PID is %d\n", getppid());
    sleep(2);
    printf("My parent's PID is %d\n", getppid());
  } else {
    // parent process
    sleep(1);
  }
  // both processes
  printf("Terminated\n");
}
```

```
$ ./orphan
My parent's PID is 2436
Terminated
$ My parent's PID is 1
Terminated
```

When the child prints for the first time, the parent is alive and the PPID is 2436. After one second the parent prints `Terminated` and exits; the shell was waiting for the parent, so it prints its prompt. After two seconds the child wakes up as an orphan adopted by `init`, so the PPID is now 1. Its output appears *after* the prompt because the shell is not waiting for it.

What Happens in the Kernel. When the parent calls `exit()`, the kernel deals with its children before `exit()` returns; the parent will never run again. Recall from Section 2.4 that every `task_struct` has a Parent pointer and a list of children. Reparenting changes two pointers (Figure 2.11): the child's Parent pointer is redirected to `init`, and the child is appended to `init`'s list of children.

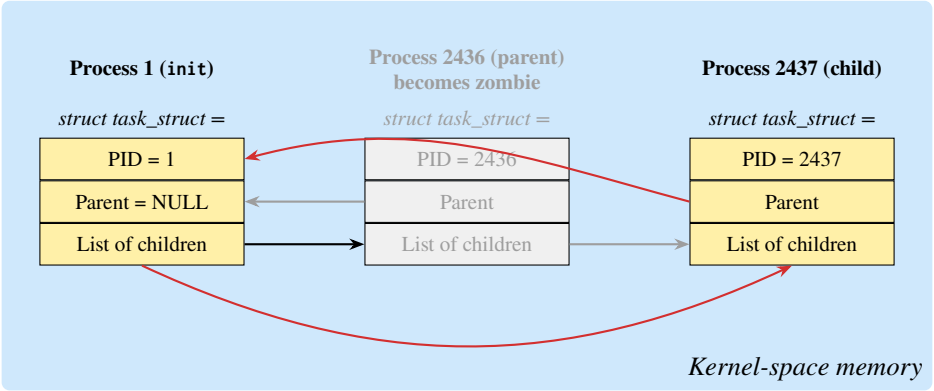


Figure 2.11: Reparenting in the kernel (after slide 107). Grey: links of the exiting parent. Red: the child's Parent pointer now points to `init`, and `init`'s list of children gains process 2437.

Reaping the Exited Parent. One question remains: the exited parent is now a zombie itself, and must be reaped by *its* parent, which is `init`. The mechanism is the one of Section 2.4: during `exit()` the kernel sends `SIGCHLD` to `init`, which calls `wait()` or `waitpid()` to reap the zombie. When the child eventually exits, its new parent `init` reaps it in the same way (Figure 2.12).

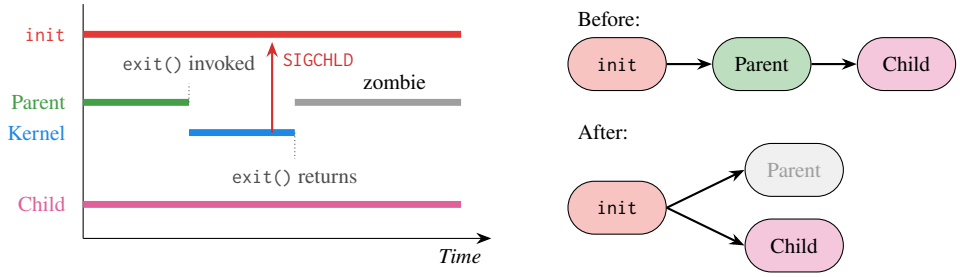


Figure 2.12: The parent's `exit()` sends `SIGCHLD` to `init`, which reaps the zombie; the child now hangs directly under `init` (after slide 108).

How `init` Reaps Zombies. Example 58 (A Loop That Does Not Work)**.**

```
int main() {
  ...
  for (;;) {
    waitpid(...);
  }
  // other work: never reached
}
```

`waitpid()` blocks, so `init` is stuck in the loop and never gets to its other work.

Example 59 (Reaping in a `SIGCHLD` Handler)**.**

```
void handler(int sig) {
  waitpid(...);
}

int main() {
  ...
  signal(SIGCHLD, handler);
  // other work
}
```

Now `init` is not blocked. It does its own work, and whenever a child exits, `SIGCHLD` interrupts it and the handler reaps the zombie.

2.6.4 Observations

Key Point 60

Processes in Linux are organized as a single tree. Thanks to reparenting every process has a parent, so all processes trace back to `init`. (Windows maintains a forest-like hierarchy; this course focuses on Unix and Linux.)

Reparenting also lets a process run without its parent shell, which is why a background job survives logout:

```
$ ./loop &
[1] 2250
$ logout
Connection to linserv1 closed.

# after logging in again
$ ps -C loop
  PID TTY          TIME CMD
 2250 ?        00:00:00 loop
```

The `TTY` column now shows `?`: `loop` no longer has a controlling terminal, but it is alive, and its parent is `init`.

2.7 Process Scheduling

2.7.1 What Scheduling Is

Computers often do several things concurrently, even with a single CPU.

Definition 61 (Multiprogramming, Multitasking, Scheduling)

Keeping several programs in memory at once is *multiprogramming*. Switching the CPU from process to process quickly, running each for a few milliseconds, is *multitasking*. Choosing which process to run next is *scheduling*, and the part of the OS that makes the choice is the *scheduler*.

2.7.2 Process States

Definition 62 (Process States)

At any moment a process is in one of the following states.

- *Ready*: runnable, but temporarily stopped to let another process run.
- *Running*: actually using the CPU at that instant.
- *Blocked* (a.k.a. sleeping or waiting): unable to run until some external event happens. The block is either *interruptible*, in which case a signal can end it, or *uninterruptible*.
- *Zombie*: exited, but not yet waited for by its parent (Section 2.4).

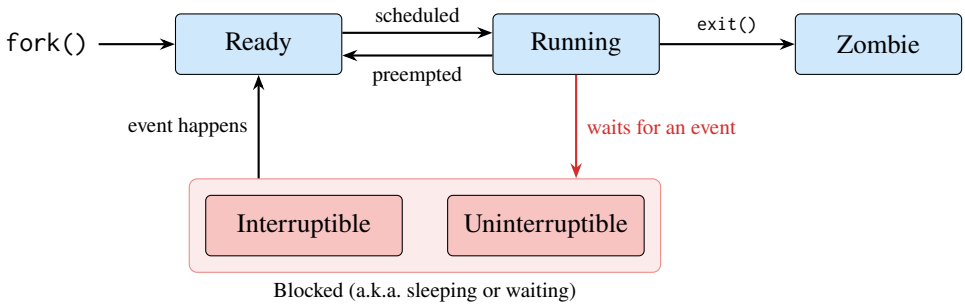


Figure 2.13: Process states (after slides 113–120).

Figure 2.13 shows the transitions.

- A process becomes *ready* when it has just been created by `fork()`, when it has run on the CPU long enough and the scheduler picks another process, or when it returns from a blocked state.
- It becomes *running* when the scheduler picks it.
- It becomes *blocked* when, for example, it calls `pause()` (Section 2.5) or reads from an I/O device. Reading a file from disk is an interruptible block: the process waits for the disk, but Ctrl-C gets it out. A process that waits for a resource and must not be disturbed meanwhile is in an uninterruptible block; this usually happens inside the kernel.

- When the external event happens, a blocked process becomes *ready* again—not running: whether it runs is still up to the scheduler.
- A *zombie* terminates for good when its parent finally waits for it.

2.7.3 Context Switching

When it is time for a process to give up the CPU, the scheduler in the kernel chooses the next process to run. Before the scheduler can take over the CPU, it must back up the register values of the current process, and the backup is stored in that process's `task_struct`.

Definition 63 (Context and Context Switch)

The *context* of a process consists of its user-space memory and its register values. If the scheduler picks a process different from the current one, the OS performs a *context switch*, which includes

- saving and restoring registers,
- switching memory maps,
- flushing and reloading the cache, . . .

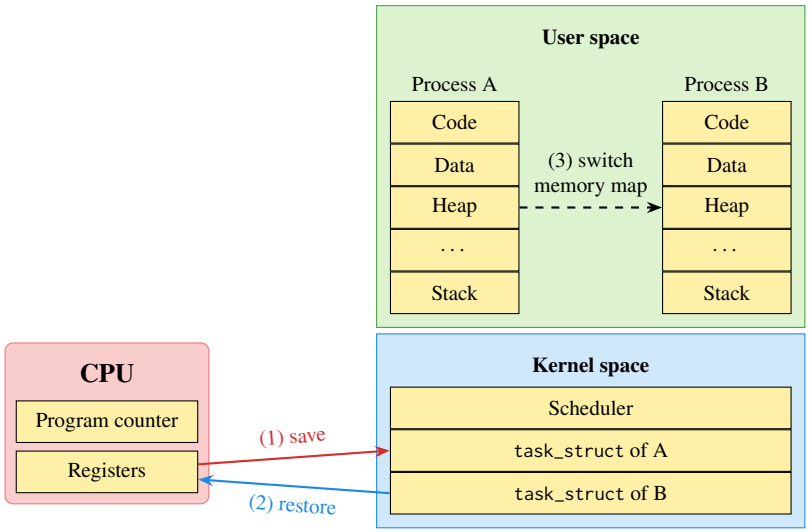


Figure 2.14: A context switch from process A to process B (after slides 121–123): (1) save A's registers in its `task_struct`; (2) restore B's registers; (3) switch to B's address space.

Key Point 64

A context switch may be expensive, and while it happens the system is not doing any useful work.

2.7.4 CPU-Bound and I/O-Bound Processes

The execution of most processes alternates between computing on the CPU and waiting for I/O.

Definition 65 (CPU-Bound and I/O-Bound)

A *CPU-bound* process spends most of its running time on the CPU: compiling a program, rendering a video, mining Bitcoin, scientific computing. The `time` command reports user time $>$ sys time. An *I/O-bound* process spends most of its running time on I/O: `/bin/ls`, downloading a file, printing an image. The `time` command reports sys time $>$ user time.

2.7.5 When to Schedule

The scheduler has to make a decision when

- a new process is created—it decides whether to run the parent or the child (recall "who runs first" in Section 2.2);
- an existing process terminates—it chooses another process, and if none is ready, runs an *idle* process;
- a process starts waiting for I/O or something else—the process is blocked, so another one must be chosen;
- a process finishes waiting for I/O—it becomes ready again, and the scheduler decides whether to run it now;

or simply periodically.

Definition 66 (Nonpreemptive and Preemptive Scheduling)

Under *nonpreemptive* scheduling, a scheduled process keeps running until it starts waiting for I/O (or something else) or voluntarily relinquishes the CPU (`man 2 sched_yield`). Under *preemptive* scheduling, a process may run for a particular period of time; when the time is up or some particular event occurs (e.g., I/O completion), it is suspended and another process may run. The hardware clock interrupt gives control of the CPU back to the scheduler.

Without the clock interrupt, a process stuck in an infinite loop would never hand the CPU back.

2.7.6 What Makes a Good Scheduler

General goals.

- *Fairness*: each process gets a fair share of the CPU.
- *Policy enforcement*: the system's policies are carried out, e.g., an administrator states that certain processes have a higher priority.
- *Balance*: all parts of the system are kept busy, e.g., by mixing CPU-bound and I/O-bound processes.

Batch systems. A batch system collects a set of jobs and processes them in batches: bank systems, data analytics, or your laundry basket, "submitted" at the end of the week. Batch jobs need no user interaction, so nonpreemptive scheduling, or preemptive scheduling with large time slices, is acceptable. The goals are

- *high throughput*: maximize the number of jobs per hour;
- *short turnaround time*: minimize the time between submission and completion;

- *high CPU utilization*: keep the CPU busy all the time.

Interactive systems. An interactive system may be a computer with an interactive user or a server with many. Preemption is essential to keep one process from denying service to others. The goals are

- *short response time*: minimize the time between request and response; interactive jobs may take precedence over background jobs;
- *proportionality*: meet users' expectations—loading a video game for minutes is acceptable, reacting a second after you press A is not.

Real-time systems. A real-time system must guarantee a response within specified time constraints, like a robot welding cars on an assembly line. The goals are

- *meeting deadlines*: avoid losing data (or a car, or a nuclear reactor); if a device produces data at a regular rate, the data-collection process must run on time;
- *predictability*: avoid quality degradation in multimedia; if the audio process runs erratically, sound quality deteriorates rapidly (jitter).

2.7.7 Scheduling Algorithms and Metrics

The algorithms below are first-come first-served (nonpreemptive), shortest job first (nonpreemptive or preemptive), round-robin (preemptive), and priority scheduling (nonpreemptive or preemptive). They are compared with the following metrics.

Definition 67 (Wait Time, Turnaround Time, Response Time)

The *wait time* of a job is the total time it is in the system but not running. Its *turnaround time* is the time from its arrival to its completion, and its *response time* is the time from its arrival to the first time it is scheduled:

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}, \qquad T_{\text{response}} = T_{\text{first run}} - T_{\text{arrival}}. \tag{2.2}$$

For a job that runs without interruption, response time equals wait time. All examples below ignore the cost of context switching.

2.7.8 First-Come, First-Served

First-come, first-served (FCFS), a.k.a. first-in first-out (FIFO), is a nonpreemptive algorithm for batch systems: jobs run to completion in order of arrival.

Example 68 (FCFS)**.** All jobs arrive at time 0. Figure 2.15 shows three runs, and the table their metrics.

	CPU requirement	Order	Wait times	Avg. wait	Avg. turnaround
(1)	$A = B = C = 10$	A, B, C	$0, 10, 20$	10	20
(2)	$A = 100, B = C = 10$	A, B, C	$0, 100, 110$	70	110
(3)	$A = 100, B = C = 10$	B, C, A	$0, 10, 20$	10	50

Runs (2) and (3) schedule the same jobs; only the order differs, yet the average turnaround time drops from 110 to 50.

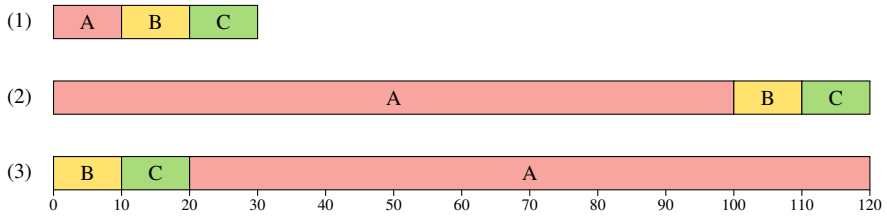


Figure 2.15: FCFS Gantt charts for the three runs of Example 68 (after slides 134–136).

The weakness of FCFS is visible in run (2): a long job at the head of the queue makes every short job behind it wait.

2.7.9 Shortest Job First

Shortest job first (SJF) always runs the job with the smallest CPU requirement. It has a nonpreemptive and a preemptive version.

With all jobs arriving at time 0, nonpreemptive SJF on $A = 100$, $B = C = 10$ produces exactly run (3) of Example 68. Things change when the short jobs arrive slightly later.

Example 69 (Nonpreemptive SJF and PSJF)**.** Job A arrives at time 0 and needs 100; jobs B and C arrive at time 10 and need 10 each (Figure 2.16).

	Wait times (A, B, C)	Avg. wait	Turnaround times (A, B, C)	Avg. turnaround
SJF	0, 90, 100	63.33	100, 100, 110	103.33
PSJF	20, 0, 10	10	120, 10, 20	50

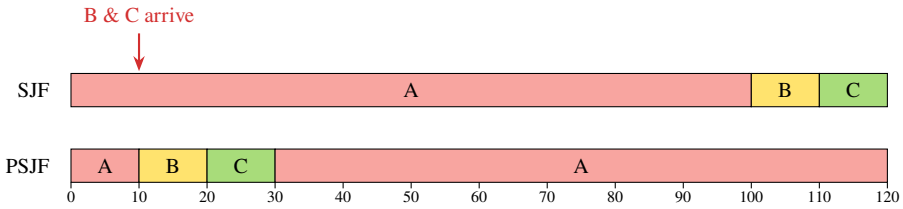


Figure 2.16: Nonpreemptive SJF and PSJF on the jobs of Example 69 (after slides 139 and 141).

Under nonpreemptive SJF, B and C arrive just after A has started, yet they must wait until A completes.

Definition 70 (Convoy Effect)

The *convoy effect* occurs when a number of relatively short consumers of a resource get queued behind a heavyweight consumer.

Preemptive shortest job first (PSJF) addresses it: whenever a new job arrives, the scheduler determines which job has the shortest *remaining* time and schedules that one, preempting the current job if necessary. In Example 69, B and C preempt A at time 10.

2.7.10 Round-Robin

PSJF looks great for batch systems, but users of interactive systems demand a short *response time*, which PSJF does not aim for.

Definition 71 (Round-Robin)

Round-robin (RR) is a preemptive algorithm for interactive systems. Each job is assigned a *time slice* (a.k.a. *quantum*), the amount of time it is allowed to run. At the end of the time slice the CPU is preempted and given to the next job. Jobs take turns in a queue called the *run queue*.

For simplicity, all jobs have the same time slice.

Example 72 (RR vs. SJF)**.** Jobs A, B, and C arrive at time 0 and need 50, 30, and 40. RR uses a time slice of 20 (Figure 2.17).

	SJF	RR
wait times (A, B, C)	$70, 0, 30$	$70, 60, 70$
turnaround times (A, B, C)	$120, 30, 70$	$120, 90, 110$
response times (A, B, C)	$70, 0, 30$	$0, 20, 40$
average wait time	33.33	66.67
average turnaround time	73.33	106.67
average response time	33.33	20
number of context switches	2	6

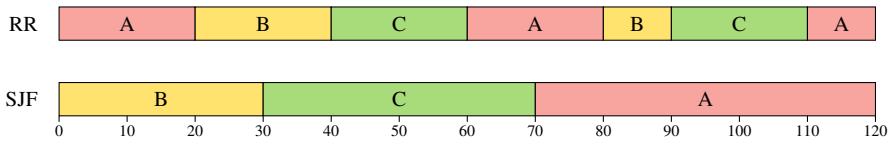


Figure 2.17: RR with a time slice of 20 vs. SJF on the jobs of Example 72 (after slides 144–146).

RR typically has worse CPU efficiency than SJF, but jobs on an RR scheduler are more responsive: each job gets the CPU from time to time, so none of them appears frozen. RR is therefore more suitable for interactive systems.

Key Point 73 (Trade-offs)

There is an inherent trade-off between performance and fairness. A fair scheduler such as RR evenly divides the CPU among active jobs on a small time scale, at the cost of turnaround time; most users run many interactive jobs and value responsiveness more than CPU efficiency.

A second trade-off comes from context switching, which is relatively slow. With a time slice of 10 ms and a context switch of 1 ms, about 10% of the time is wasted; with a time slice of 100 ms, less than 1% is. A short time slice means many context switches and low CPU efficiency; a long time slice means poor response, and the system feels sluggish.

2.7.11 Priority Scheduling

Definition 74 (Priority Scheduling)

In *priority scheduling*, each job is assigned a priority, and the scheduler always chooses the job with the highest priority to run. With *static* priorities a job receives a fixed priority when it is submitted—a background email process, say, gets a lower priority than a real-time video game. With *dynamic* priorities a job's priority may change throughout its life.

Static priority scheduling. Jobs are kept in one queue per priority level, and each queue is served round-robin (Figure 2.18). Jobs with priority 3 are scheduled first; they should be short-lived, to prevent *starvation* of lower-priority jobs. When no job of priority 3 is left, jobs of priority 2 are scheduled, and so on. When a higher-priority job appears, the running job may or may not be preempted, depending on whether the scheduler is preemptive.

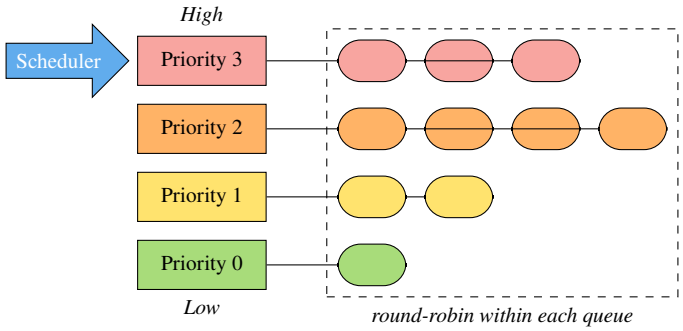


Figure 2.18: Static priority scheduling with four priority levels (after slides 150–152).

Static priorities have serious limitations.

- High-priority jobs may run for a prolonged period, even indefinitely, and low-priority jobs may starve. Rumor has it that when the IBM 7094 at MIT was shut down in 1973, a low-priority process submitted in 1967 had not yet run.
- Static priorities do not distinguish CPU-bound from I/O-bound jobs. An I/O-bound job spends most of its time waiting for I/O; when it does want the CPU, it is best to schedule it immediately so that it can start its next I/O request. That way I/O proceeds in parallel with another process's computation.

Dynamic priority scheduling. There is no standard way to assign priorities dynamically. Recalling the trade-off between efficiency and responsiveness, we would ideally

- optimize turnaround time: run shorter jobs first, and give CPU-bound jobs a large time slice to reduce context switching;
- make the system responsive to interactive users: we cannot give every job a large time slice, and we want to minimize response time.

2.7.12 Multilevel Feedback Queue

The *multilevel feedback queue* (MLFQ) is a form of dynamic priority scheduling in which each priority level has its own policy. In the running example there are four levels, each served round-robin, with longer time slices at lower priority:

Priority	Policy
3 (highest)	round-robin, time slice 10
2	round-robin, time slice 20
1	round-robin, time slice 40
0 (lowest)	round-robin, time slice 80

All jobs start at the highest priority. A job that uses up its time slice drops one level; a job that gives up the CPU before its time slice is up stays at the same level.

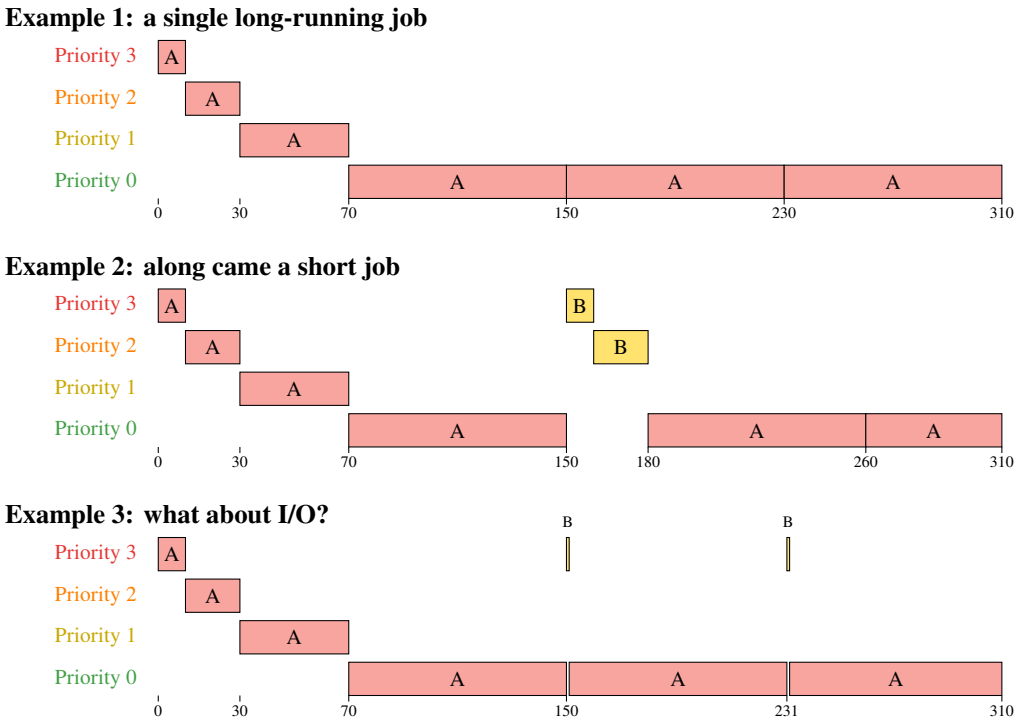


Figure 2.19: MLFQ with time slices 10, 20, 40, 80 (after slides 156–158). Tick 10 is omitted for space.

Example 75 (MLFQ at Work)**.** Figure 2.19 shows three scenarios.

1. *A single long-running job.* A uses up its slice at every level and sinks to priority 0, where it receives slices of 80. Long-running jobs get a large time slice, which reduces context switching.
2. *Along came a short job.* B arrives at time 150 and needs 30. It enters at priority 3, preempting A, and finishes within two levels at time 180, after which A resumes. Short jobs run first—MLFQ approximates SJF without knowing job lengths in advance.
3. *What about I/O?* B is I/O-bound: it uses the CPU for 1 unit at a time and then waits for I/O. Since it always gives up the CPU before its slice is up, it stays at priority 3 and gets the CPU as soon as it wants it. I/O-bound jobs have a higher priority to access the CPU.

The scheme has two limitations. Long-running jobs may starve if there are too many interactive jobs. And a program may change its behavior over time: a job that computes for a long time when it starts and becomes interactive afterwards has already sunk to the bottom and is punished forever. Both are fixed by a *priority boost*: after some time period, move all jobs to the highest priority.

Key Point 76 (The Rules of MLFQ)

1. If Priority$(A) >$ Priority(B), A runs (B does not).
2. If Priority$(A) =$ Priority(B), A and B run round-robin using the time slice of their queue.
3. When a job enters the system, it is placed at the highest priority (the topmost queue).
4. Once a job uses up its time slice at a given level, its priority is reduced (it moves down one queue).
5. After some time period, move all the jobs in the system to the topmost queue.

MLFQ observes how jobs behave over time and prioritizes them accordingly. It delivers excellent overall performance, similar to SJF and PSJF, for short-running interactive jobs, and it is fair and makes progress for long-running CPU-intensive workloads. Many modern operating systems therefore use a form of MLFQ as their base scheduler.

Key Point 77 (Summary)

There is no best or standard scheduling algorithm, partly because we cannot predict the CPU requirement of a process, we do not even know whether a job will eventually terminate, and online scheduling is NP-hard. Linux has used the *Completely Fair Scheduler* (CFS), a dynamic priority algorithm based on red-black trees, since kernel 2.6.23.

2.8 Interprocess Communication

2.8.1 What IPC Is and Why We Need It

Definition 78 (Interprocess Communication)

Processes often need to communicate with one another. Any mechanism that lets them do so is called *interprocess communication* (IPC).

Several forms of IPC have already appeared in these notes:

- *signals*: `kill -STOP 2250` tells process 2250 to stop (Section 2.5);
- *pipelines*: in `ls | less`, the output of `ls` becomes the input of `less`;
- *files*, *sockets*, and *shared memory*.

There are three reasons for IPC.

- *To share information.* This is the obvious one.
- *To reuse software.* Small programs that each do one job can be combined into a larger one. The following pipeline is a spell checker for a Wikipedia page:

```
curl "https://en.wikipedia.org/wiki/Pipeline_(Unix)" | sed 's/[^a-zA-Z ]/ /g' |
  tr 'A-Z ' 'a-z\n' | grep '[a-z]' | sort -u |
  comm -23 - <(sort /usr/share/dict/words) | less
```

  `curl` downloads the page; `sed` replaces every character that is not a letter or a space with a space; `tr` lowercases the letters and turns each space into a newline, so there is one word per line; `grep` drops the empty lines; `sort -u` sorts the words and removes duplicates; `comm -23` prints the lines that appear only in its first input (the page's words) and not in its second (the sorted dictionary), i.e., the misspelled or unknown words; and `less` pages through them. None of these programs knows anything about spell checking.

- *To speed up computation.* A job can be divided into tasks that run as separate processes in parallel, after which the results are combined. MapReduce is the standard example (CSCI-UA.0476 and CSCI-GA.2436 cover big-data systems).

2.8.2 Two Ways to Do IPC

Definition 79 (Shared Resource and Message Passing)

With a *shared resource*, processes communicate by reading and writing data that all of them can access. With *message passing*, processes communicate by sending data to one another; the data is copied from the sender to the receiver.

A pipe is message passing: `ls` hands data to the kernel, which hands it to `less`. Shared memory is a shared resource: both processes read and write the same bytes, and the kernel is not involved in each access. The two subsections below look at one example of each.

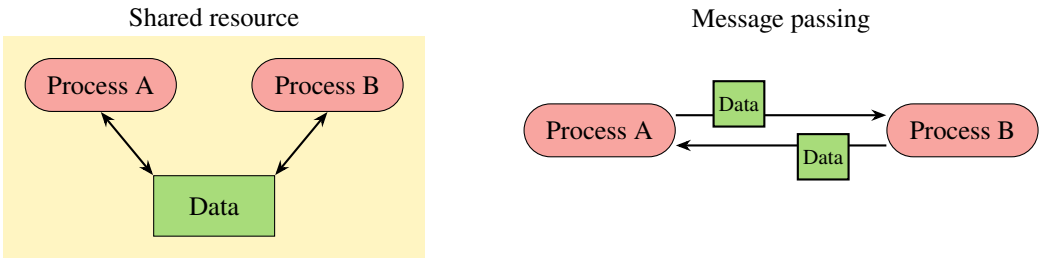


Figure 2.20: The two models of IPC (after slide 166).

2.8.3 Case Study: Pipes

The programmer's view. The `pipe()` system call creates a pipe and returns two file descriptors, `pipefd[0]` and `pipefd[1]`. Data written to `pipefd[1]` (the write end) can be read from `pipefd[0]` (the read end), in the same order. In `ls | less`, `ls` writes its output to the write end, and `less` reads its input from the read end.

Definition 80 (Producer-Consumer Model)

When one process produces data and another consumes it, through a buffer between them, the arrangement is called the *producer-consumer model*. A pipe is a producer-consumer model: `ls` is the producer and `less` the consumer.

The kernel's view. The buffer of a pipe lives in the kernel. `ls` calls `write()` on the write end, which copies its data into the kernel buffer; `less` calls `read()` on the read end, which copies data out of it. The buffer has a fixed size, and the kernel handles its two boundary cases:

- if the pipe is full, `write()` blocks until `less` has read something;
- if the pipe is empty, `read()` blocks until `ls` has written something.

Definition 81 (Process Synchronization)

Coordinating processes so that each waits for the others when necessary, here a writer waiting for free space and a reader waiting for data, is called *process synchronization*.

So the producer never overwrites unread data and the consumer never reads garbage, without either program doing anything special. `man 7 pipe` has the details.

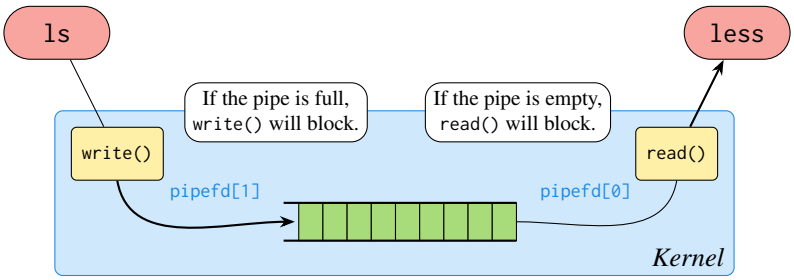


Figure 2.21: `ls | less`: the pipe is a bounded buffer in the kernel, and the kernel blocks `write()` on a full pipe and `read()` on an empty one (after slides 167–168).

2.8.4 Shared Memory

Definition 82 (Shared Memory)

Shared memory is a region of memory created by the kernel that can be mapped into the address spaces of several processes. It is visible to all processes in the system (a pipe, by contrast, is visible only to the two processes at its ends) and accessible by all of them, although system calls can change its ownership and permissions. `man 7 shm_overview` has the details.

Once the region is mapped, processes read and write it with ordinary loads and stores; no system call is needed per access. That makes shared memory the fastest form of IPC, but it also means the kernel no longer takes part in each access. With a pipe, the kernel provided synchronization for free; with shared memory, it is up to the processes to coordinate. What could possibly go wrong?

Example 83 (Two Processes Incrementing a Shared Counter)**.** `shm.c` creates a 4-byte shared counter, forks, and lets both processes increment it a million times. The parent then waits for the child and prints the counter.

```
int main() {
  // create a 4-byte shared memory
  int *count = mmap(NULL, 4, PROT_READ | PROT_WRITE,
                    MAP_SHARED | MAP_ANONYMOUS, -1, 0);
  pid_t pid = fork();
  for (int i = 0; i < 1000000; ++i) {
    ++*count;
  }
  if (pid) {
    wait(NULL);
    printf("count = %d\n", *count);
  }
}
```

`mmap()` asks the kernel for a new mapping: `NULL` lets the kernel choose the address, `4` is the length, and the protection allows reading and writing. `MAP_ANONYMOUS` means the region is not backed by a file (hence the descriptor -1 and offset 0), and the memory starts zeroed. `MAP_SHARED` is the important flag: the child inherits the mapping across `fork()`, and unlike ordinary memory, which `fork()` shares copy-on-write (Section 2.2), writes to a shared mapping are seen by both processes.

The expected output is $2\,000\,000$. The actual output is different every time:

```
$ ./shm
count = 1434181
$ ./shm
count = 1062142
$ ./shm
count = 1256405
```

Why updates are lost. `++*count` looks like one operation, but it is not. Compiled without optimization (`gcc -S shm.c`), it becomes

```
    movq    -16(%rbp), %rax     # a = count      (load the pointer)
```

```
movl    (%rax), %eax        # a = *a        (load the counter)
leal    1(%rax), %edx       # d = a + 1
movq    -16(%rbp), %rax     # a = count
movl    %edx, (%rax)        # *a = d        (store the counter)
```

Ignoring the pointer loads, it is three steps: `a = *count; d = a + 1; *count = d;`. The value lives in a register between the load and the store, and a context switch can occur between any two instructions. If it occurs in that window, the other process's increment is overwritten (Figure 2.22): both processes read 0, both compute 1, both store 1. Two increments, one effect. Over two million increments this happens hundreds of thousands of times. On a multiprocessor, the parent and the child run truly in parallel, and the same interleaving occurs without any context switch.

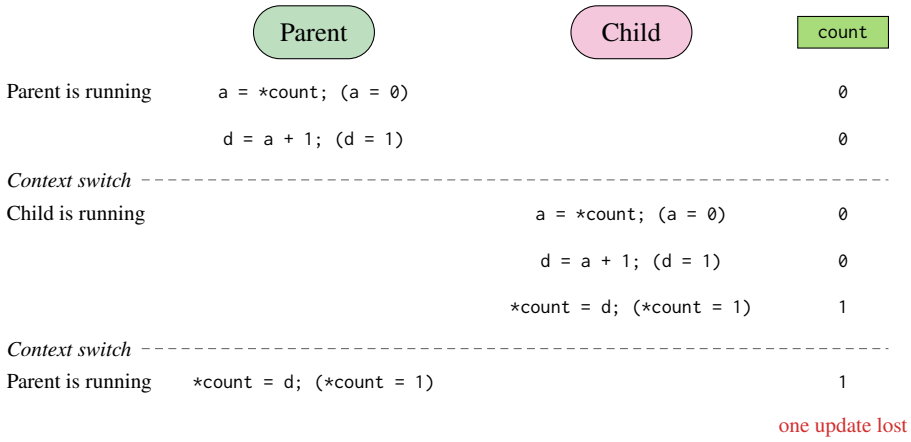


Figure 2.22: A context switch between the load and the store of `++*count` loses one of two increments (after slide 174).

2.8.5 Race Conditions

Definition 84 (Race Condition)

A *race condition* occurs when the result of a computation depends on the timing of the execution, i.e., on the particular order in which processes access a shared resource. When the shared resource is memory, and at least one of the concurrent accesses is a write, it is more specifically a *data race*.

Race conditions are always bad.

- Compiler optimizations may produce surprising output when code has data races. The compiler assumes that no one else modifies `*count` during the loop, so it is free to keep the counter in a register or to rewrite the loop entirely. Data races are undefined behavior (the "nasal demons" of C folklore; see *Schrödinger's Code* for an introduction).
- The computation is nondeterministic, so debugging is no fun at all. A *Heisenbug* is a bug that disappears or changes its behavior when you try to observe it: adding a `printf` or running under a debugger changes the timing, and the race no longer shows.

Key Point 85

A pipe synchronizes for you; shared memory does not. Whenever processes share memory, they must coordinate their accesses themselves, which is the subject of the next section.

2.9 Critical Sections and Mutual Exclusion

2.9.1 Mutual Exclusion and Critical Sections

Definition 86 (Mutual Exclusion)

To avoid race conditions, we need *mutual exclusion*: if one process is accessing a shared resource, the other processes must be excluded from accessing the same resource.

The race condition is the *problem*; mutual exclusion is a *requirement* that avoids it. The requirement has a cost: while one process is excluded, it cannot make progress, so mutual exclusion may hinder the performance of parallel computations.

Definition 87 (Critical Section)

A *critical section* (a.k.a. critical region) is a piece of code that accesses a shared resource. It is preceded by a *section entry*, which declares the entry of the critical section, and followed by a *section exit*, which declares its exit (Figure 2.23).

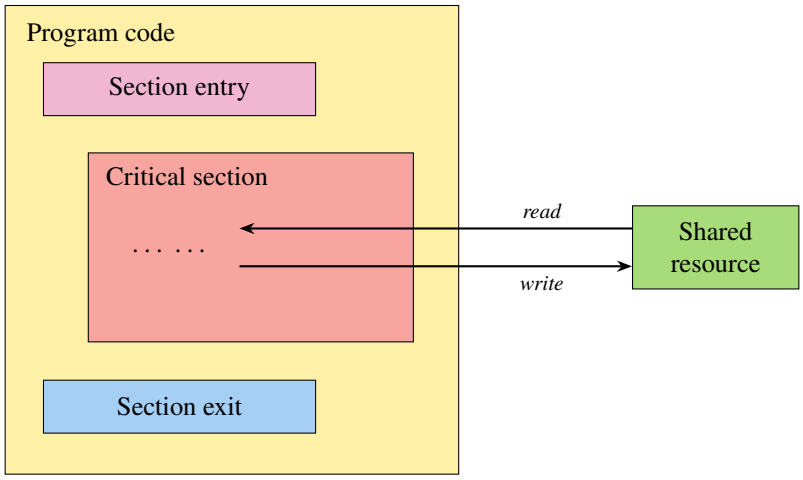


Figure 2.23: A critical section is the code between a section entry and a section exit that reads or writes a shared resource (after slide 177).

The slides compare a critical section to a restroom with a single key: when a process enters its critical section, it locks the door; when it exits, it unlocks the door; the processes queued outside are blocked. Different people do different things inside, and likewise different processes may have *different* critical sections that protect the same resource.

Key Point 88 (Critical Sections Are Code)

- A critical section is a piece of *code*, not a shared resource.
- A critical section should be *as tight as possible*. If the entire program were declared one big critical section, the processes would run one after another and all parallelism would be lost.
- A critical section may access *multiple* shared resources. Two critical sections need mutual exclusion if the sets of shared resources they access intersect.

2.9.2 What a Good Solution Must Satisfy

Key Point 89 (Four Requirements)

1. *Mutual exclusion.* No two processes may be simultaneously inside their critical sections.
2. *No assumptions about speed or CPUs.* The solution cannot depend on how long a process spends inside its critical section or on the number of CPUs in the system.
3. *Progress.* No process running outside its critical section may block other processes. Otherwise all processes could end up blocked while no process is inside its critical section.
4. *Bounded waiting.* No process should have to wait forever to enter its critical section, i.e., no process starves to death.

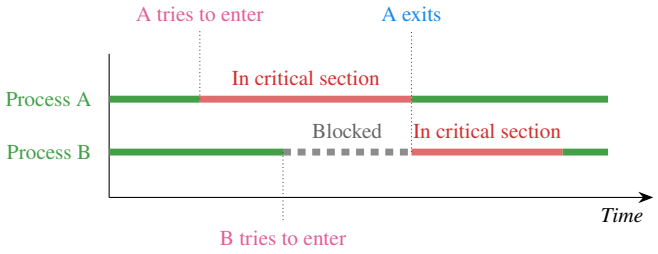


Figure 2.24: Mutual exclusion in action: B tries to enter while A is inside, so B blocks in its section entry until A exits (after slide 181).

It is always the *section entry* that gets blocked, not the critical section itself (Figure 2.24). The whole problem is therefore how to implement the section entry and the section exit. The rest of this section tries six designs and judges each by two questions: is it *correct* (does it guarantee mutual exclusion?), and is it a *good* solution (does it satisfy the other requirements, and does it perform well?).

2.9.3 Attempt 1: Disabling Interrupts

Section entry disables interrupts; section exit enables them. With interrupts disabled, the timer interrupt cannot fire, so no context switch can occur, and while one process is in its critical section, the CPU will not be switched to another process.

- *Correct?* On a single-processor system, yes. On a multiprocessor system, *no*: disabling interrupts affects only the current CPU, and the other CPUs can still access the shared resource.

- *Good?* Even on a single processor it is a terrible idea to let user processes enable and disable interrupts at will: a process that never re-enables them owns the machine. Within the OS kernel itself, however, it is often convenient to disable interrupts for a few instructions.

2.9.4 Attempt 2: A Lock Variable

Use a shared variable `lock`: 0 means free, 1 means some process is inside.

```
int lock = 0;  // shared

// section entry
1  while (lock == 1)
2    ;  // busy waiting
3  lock = 1;
   ... critical section ...
// section exit
   lock = 0;
```

- *Correct? No.* It suffers from exactly the same race condition as `shm.c`. Process A finds `lock == 0` on line 1 and is switched out before line 3; process B also finds `lock == 0`, sets it, and enters; A resumes, sets `lock = 1`, and enters too. The test and the set are two separate steps, just like the load and the store of `++*count`.
- *Good, even if it worked?* No, for two reasons.

Definition 90 (Busy Waiting)

Repeatedly testing a variable until some value appears is called *busy waiting*, or *spinning*. It wastes CPU time: the spinning process uses the CPU without doing anything useful.

Definition 91 (Priority Inversion)

Priority inversion occurs when a high-priority process has to wait for a low-priority process to release a resource. With busy waiting and a priority scheduler it can last forever: if the low-priority process holds the lock and the high-priority one spins, the scheduler always runs the high-priority process, so the lock holder never gets the CPU to release the lock.

2.9.5 Attempt 3: Strict Alternation

Let the processes take turns, using a shared variable `turn` that says whose turn it is:

```
// Process 0
1  for (;;) {
2    while (turn != 0)
3      ;  // busy waiting
4    critical_section();
5    turn = 1;
6    noncritical_section();
7  }
```

```
// Process 1
1  for (;;) {
2    while (turn != 1)
3      ;  // busy waiting
4    critical_section();
5    turn = 0;
6    noncritical_section();
7  }
```

with `int turn = 0;` shared. The critical sections are executed in alternating order: 0, 1, 0, 1, . . .

- *Correct?* Yes, it avoids all races: `turn` is only ever written by the process that holds the turn, so a process can enter only when the other has handed the turn over.
- *Good?* No. It violates Requirement 3 (progress). Suppose process 0 finishes its critical section, sets `turn = 1`, finishes its short noncritical section, and wants to enter again, while process 1 is busy in a long noncritical section. Process 0 must wait until process 1 has gone through its critical section, so a process outside its critical section blocks another. Neither process can enter its critical section twice in a row.

2.9.6 Attempt 4: Peterson's Algorithm

Peterson's algorithm (1981) improves strict alternation by combining it with lock variables: each process announces its *intent* to enter, and `turn` only breaks ties.

```
int intent[2] = {0, 0};  // who wants to enter critical section?
int turn;                // whose turn is it?

1  void section_entry(int process) {  // process is 0 or 1
2    int other = 1 - process;         // the other process is 1 or 0
3    intent[process] = true;          // I intend to enter critical section
4    turn = other;                    // make it the other process's turn
5    while (intent[other] && turn == other)  // I'm a gentleman!
6      ;                              // busy waiting while it's not my turn
7  }

1  void section_exit(int process) {
2    intent[process] = false;         // simply undo my intent
3  }
```

The processes act as gentlemen: "if you want to enter, I'll let you go first." A process waits only while the other one *both* wants to enter *and* has the turn. If the other is not interested, it enters immediately, which fixes the progress problem of strict alternation.

Example 92 (A Run of Peterson's Algorithm)**.**

Process 0	Process 1
`intent[0] = true`, `turn = 1`	
	`intent[1] = true`, `turn = 0`
`turn` is not `other`: in critical section	
	`turn == 0` and `intent[0]`: busy waiting...
`intent[0] = false`	
	`intent[0]` is false: in critical section
	`intent[1] = false`

Both processes want to enter, and process 1 wrote `turn` last, so process 1 is the one that waits.

Why can both never be inside? Suppose they were, and say process 1 wrote `turn` last, setting it to 0. Process 0 set `intent[0]` before its own, earlier, write to `turn`, so when process 1 then evaluated line 5 it saw `intent[0]` true and `turn == 0`, and could not have passed. In general, whichever process writes `turn` last gives the turn away and waits until the other exits.

- *Correct?* Yes on early hardware, where writes to `intent[]` and `turn` became visible to the other CPU immediately and in program order. *Not anymore* on modern hardware, because of relaxed memory consistency models: a CPU may let its own later load of `intent[other]` complete before its earlier store to `intent[process]` is visible to the other CPU, so both processes can read the other's intent as false and both enter.
- *Good?* It still busy-waits, and it is for two processes only. With a little hardware support, the solution can be much simpler.

2.9.7 Attempt 5: Spinlocks

Modern CPUs all have instructions that are guaranteed to execute atomically. The simplest is *test-and-set*: conceptually, it executes the following code without interruption, returning the old value pointed to by `ptr` and simultaneously updating it to `new`.

```
int test_and_set(int *ptr, int new) {
  // this code executes atomically
  int old = *ptr;
  *ptr = new;
  return old;
}
```

It is the lock variable of Attempt 2 with the test and the set fused into one step, which is exactly what Attempt 2 was missing:

```
int lock = 0;  // 0: available, 1: held

void section_entry(int *lock) {
  while (test_and_set(lock, 1) == 1)
    ;  // busy waiting
}

void section_exit(int *lock) {
  *lock = 0;
}
```

If `test_and_set` returns 0, the lock was free and the caller has just taken it; if it returns 1, the lock was held, setting it to 1 again changed nothing, and the caller retries.

Definition 93 (Spinlock)

A *spinlock* is a lock built upon CPU instructions that guarantee atomic operation, such as test-and-set, compare-and-swap, fetch-and-add, or load-linked/store-conditional, where a process waiting for the lock spins in a loop.

- *Correct?* Yes, on a preemptive scheduler. On a nonpreemptive one, a process spinning on a single CPU would never let the holder run. Beware of priority inversion.
- *Good?*
  - *Simplicity:* yes.
  - *Fairness:* no guarantees. Under contention a process may spin forever while others keep winning the lock, so it can starve.
  - *Performance on a single processor:* bad. With N processes contending for a lock, $(N-1)/N$ of the CPU time is wasted on spinning, since only the holder makes progress.
  - *Performance on a multiprocessor:* effective if the number of processes is about the number of CPUs and the critical section is short, because then the holder is running on another CPU and will release the lock soon.

Attempt 5.1: less spinning. One change makes the spinlock less wasteful: instead of spinning, give up the CPU.

```
void section_entry(int *lock) {
  while (test_and_set(lock, 1) == 1)
    sched_yield();  // give up the CPU
}
```

`sched_yield()` (Section 2.7) moves the caller to the back of the run queue. This run-and-yield approach is better than spinning but still costly: before the process holding the lock gets to run again, every other contending process still has to run and yield, and each of those context switches is expensive. What we want is to *block* instead of spinning, and be woken up when the lock is released. It is time to introduce a new OS primitive.

2.9.8 Attempt 6: Semaphores

Definition 94 (Semaphore)

A *semaphore* is an object with a non-negative integer value, which must be initialized before it is used. It supports two operations, each of which executes atomically:

- `down(sem)`: if the value is positive, decrement it; otherwise, block on the semaphore;
- `up(sem)`: if some process is blocked on the semaphore, let one such process proceed; otherwise, increment the value.

`man 7 sem_overview` describes the POSIX version.

```
void down(semaphore *sem) {
  // atomic operation
  if (*sem > 0) {
    *sem = *sem - 1;
  } else {
    block on sem;
  }
}
```

```
void up(semaphore *sem) {
  // atomic operation
  if (some process is blocked on sem) {
    let one such process proceed
  } else {
    *sem = *sem + 1;
  }
}
```

The value of a semaphore counts available "permits". A process that finds none is put to sleep by the kernel, using no CPU at all, and the matching `up()` hands the permit directly to one sleeper instead of incrementing the value. The name comes from the railway signal; `down` and `up` are Dijkstra's P and V.

A semaphore initialized to 1 is a *binary semaphore*, and it is a lock:

```
semaphore sem = 1;  // init a binary semaphore

down(&sem);   // section entry
   ... critical section ...
up(&sem);     // section exit
```

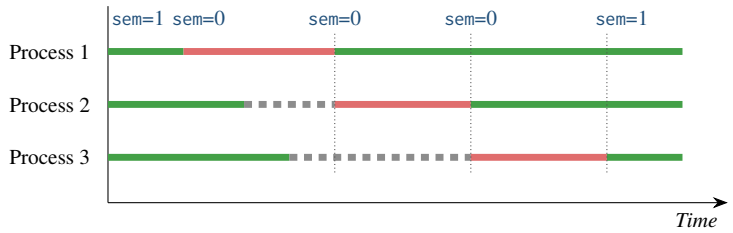


Figure 2.25: A binary semaphore as a lock (after slide 194). Red: in the critical section; dashed: blocked in `down()`. Each `up()` hands the permit straight to a blocked process, so `sem` stays 0 until the last process leaves.

Definition 95 (Mutex)

A lock that guarantees mutual exclusion is called a *mutex* (for "mutual exclusion"). A binary semaphore is one way to implement it.

- *Correct?* Yes.
- *Good?* Yes. Waiting processes block instead of spinning, and the kernel decides whom to wake.

Semaphores are more powerful than mutual exclusion. They can also realize *process synchronization*: coordinating a set of processes so that together they produce meaningful output, as the kernel did for the pipe in Section 2.8. A semaphore whose value may exceed 1 is called a *counting semaphore*; the next section uses counting semaphores to count free and filled buffer slots.

Attempt	Correct?	Good?
1. Disable interrupts	single CPU only	no: user processes must not control interrupts
2. Lock variable	no: same race	no: busy waiting, priority inversion
3. Strict alternation	yes	no: violates progress
4. Peterson	not on modern hardware	no: busy waiting, two processes only
5. Spinlock	yes (preemptive scheduler)	only for short sections on multiprocessors
6. Semaphore	yes	yes

2.10 Classical IPC Problems

Four classical problems model the situations in which processes share resources. Each is solved below with the semaphores of Section 2.9.

Problem	What it models
producer-consumer	access to a bounded buffer
dining philosophers	processes competing for exclusive access to a limited number of resources (e.g., I/O devices)
readers and writers	access to a database
sleeping barber	a queueing situation ("please stay on the line. . . ")

2.10.1 The Producer-Consumer Problem

Definition 96 (Producer-Consumer Problem)

In the *producer-consumer problem*, a.k.a. the *bounded-buffer problem*, two processes share a fixed-size buffer. The *producer* inserts data at the tail of the buffer, and the *consumer* removes data from the head.

Sounds familiar? It is a pipe (Section 2.8). The kernel solves it for pipes; the question now is how to implement it ourselves with shared memory.

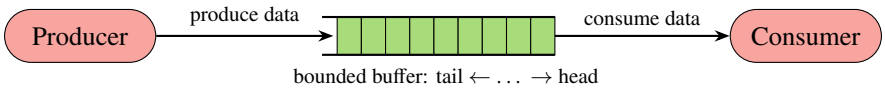


Figure 2.26: The producer-consumer problem (after slide 198).

A solution must meet three requirements.

1. *Synchronization when the buffer is full.* If the producer wants to insert an item but the buffer is full, the producer should block, and the consumer should wake it up after it has consumed an item.
2. *Synchronization when the buffer is empty.* If the consumer wants to remove an item but the buffer is empty, the consumer should block, and the producer should wake it up after it has produced an item.
3. *Mutual exclusion.* No two processes can access the shared buffer at the same time.

Solution with three semaphores. One semaphore per requirement:

```
semaphore mutex = 1;           // controls access to critical section
semaphore empty = BUFFER_SIZE; // counts empty buffer slots
semaphore filled = 0;          // counts filled buffer slots
```

```
1  void producer() {
2    int item;
3
4    for (;;) {
5      item = produce_item();
6      down(&empty);  // decrement empty count
7      down(&mutex);  // enter critical section
8      insert(item);  // put new item in buffer
9      up(&mutex);    // exit critical section
10     up(&filled);   // increment filled count
11   }
12 }
```

```
1  void consumer() {
2    int item;
3
4    for (;;) {
5      down(&filled);    // decrement filled count
6      down(&mutex);     // enter critical section
7      item = remove();  // take item from buffer
8      up(&mutex);       // exit critical section
9      up(&empty);       // increment empty count
10     consume_item(item);
11   }
12 }
```

Why three semaphores? Because there are three requirements.

- `mutex` is a binary semaphore that guarantees mutual exclusion on the buffer.
- `empty` and `filled` are counting semaphores used for synchronization. `empty` is the number of empty slots: the producer blocks on it when it is 0, i.e., when the buffer is full. `filled` is the number of filled slots: the consumer blocks on it when it is 0, i.e., when the buffer is empty.

Each process `down()`s the semaphore that counts what it needs and `up()`s the one that counts what it creates. The invariant is `empty` + `filled` = `BUFFER_SIZE`, up to the one item that is in flight between a process's `down()` and its `up()`. Note that `produce_item()` and `consume_item()` are outside the critical section: the critical section is kept as tight as possible.

Swapping lines 6 and 7 of the producer. Can the producer take the `mutex` first and then wait for an empty slot? Start with `mutex = 1`, `empty = N`, `filled = 0`, and let the producer run alone first (Figure 2.27).

1. The producer runs lines 4–11 N times and fills the buffer: `mutex = 1`, `empty = 0`, `filled = N`.
2. It produces one more item (line 5), takes the mutex (line 7, `mutex = 0`), and blocks on `empty` (line 6) *while holding the mutex*.
3. The consumer takes an item's worth of `filled` (line 5, `filled = N-1`) and blocks on `mutex` (line 6).

Now nobody can proceed. The producer waits for the consumer to `up(&empty)`, which the consumer will do only after it gets the mutex; the consumer waits for the producer to `up(&mutex)`, which the producer will do only after it gets an empty slot.

Definition 97 (Deadlock)

A *deadlock* is a situation in which every process of a set is blocked waiting for something that only another process of the set can provide, so none of them can ever proceed. It happens when there is a *circular wait*.

The slides' picture is two people in two restroom stalls: one is waiting for the other's paper, the other is waiting for the first one to come out, and neither will move first. The lesson for semaphores: never block on a synchronization semaphore while holding the mutex. Deadlocks get their own treatment at the end of this deck.

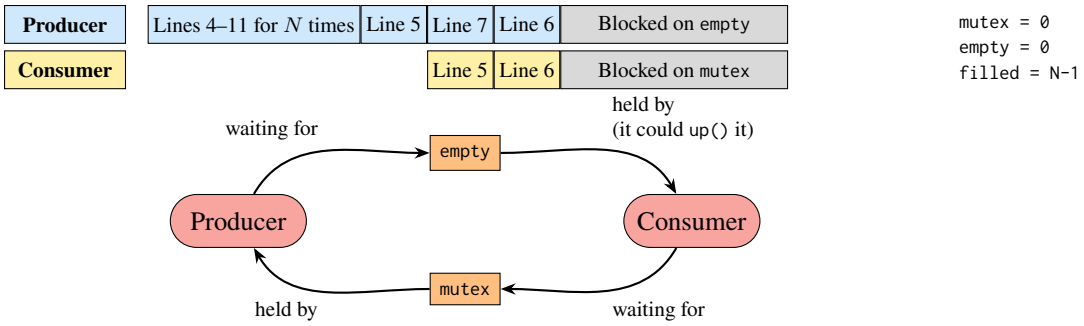


Figure 2.27: Swapping lines 6 and 7 of the producer: the execution that gets stuck (top), and the circular wait behind it (bottom) (after slides 205–211).

Key Point 98 (Producer-Consumer, Summary)

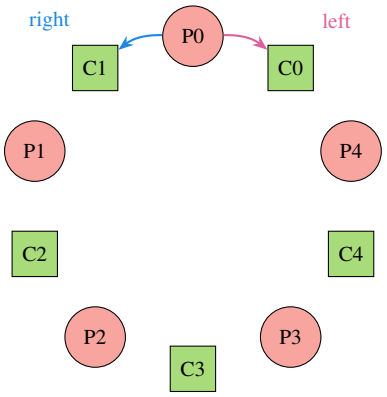
A binary semaphore is used as a lock to protect the critical sections (*mutual exclusion*). Two counting semaphores monitor the status of the buffer (*synchronization*). The order of the `down()`s matters.

If the buffer were *unbounded*, the producer would never have to wait, so the `empty` semaphore and the producer's `down(&empty)` disappear; `mutex` and `filled` are still needed.

2.10.2 The Dining Philosophers Problem

Definition 99 (Dining Philosophers Problem)

Five philosophers sit around a table with five bowls of instant ramen, and between each pair of neighbors lies one chopstick. A philosopher does only two things in their entire life: eating and thinking. To eat, a philosopher needs exactly two chopsticks, the ones on their left and right. Design a protocol so that they can all enjoy their life.



For each philosopher i:
left chopstick $= i$
right chopstick $= (i+1) \bmod N$
`LEFT` neighbor $= (i+N-1) \bmod N$
`RIGHT` neighbor $= (i+1) \bmod N$

States: `THINKING`,
`HUNGRY` (trying to get chopsticks),
`EATING` (critical section).

Figure 2.28: Modeling the dining philosophers, $N = 5$ (after slide 216).

There are two requirements.

1. *Mutual exclusion.* Without it, while you are eating, the two philosophers sitting next to you will steal your chopsticks.

2. *Synchronization.* The solution should avoid any potential deadlock or starvation.

Attempt 1: a mutex for each chopstick. Model each chopstick as a binary semaphore, with all `chopsticks[i]` initialized to 1 (each chopstick starts on the table):

```
semaphore chopsticks[N];  // all initialized to 1

void take_chopstick(int i) { down(&chopsticks[i]); }
void put_chopstick(int i)  { up(&chopsticks[i]); }

1  void philosopher(int i) {
2    for (;;) {
3      think();
4      take_chopstick(i);
5      take_chopstick((i + 1) % N);
6      eat();                        // critical section
7      put_chopstick((i + 1) % N);
8      put_chopstick(i);
9    }
10 }
```

This deadlocks. If all five philosophers become hungry at about the same time and each executes line 4, every philosopher holds their left chopstick, and every one of them blocks on line 5 waiting for the right chopstick, which is their right neighbor's left chopstick (Figure 2.29). It is the circular wait of Figure 2.27, with five processes instead of two.

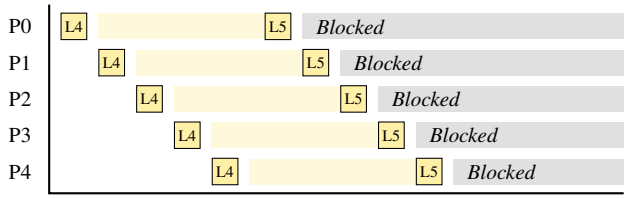


Figure 2.29: Attempt 1: each philosopher takes the left chopstick (line 4), then blocks forever on the right one (line 5) (after slide 218).

Attempt 2: a mutex for each chopstick, with backoff. To avoid holding one chopstick while blocking on the other, use a `try_to_take_chopstick()` that either succeeds and takes the chopstick, or fails and does not take it. On failure, put the left chopstick back, wait, and try again:

```
1  void philosopher(int i) {
2    for (;;) {
3      think();
4      take_chopstick(i);
5      while (try_to_take_chopstick((i + 1) % N) == FAILED) {
6        put_chopstick(i);
7        sleep(1);           // wait for 1s and try again
8        take_chopstick(i);
9      }
10     eat();                // critical section
11     put_chopstick((i + 1) % N);
```

```
12     put_chopstick(i);
13   }
14 }
```

There is no deadlock any more, but if the philosophers run in lockstep, they all take their left chopstick (line 4), all fail on the right one (line 5), all put the left one back (line 6), all sleep for exactly one second (line 7), and do it all over again, forever.

Definition 100 (Livelock)

A *livelock* is a situation in which no process is blocked, yet none of them makes any progress. It is a kind of starvation (literally, here).

Attempt 3: random backoff. Replace `sleep(1)` by `sleep(random())`. The philosophers now wait for different amounts of time, so sooner or later one of them retries while its neighbor is still asleep, takes both chopsticks, and eats. This is good in most applications; in fact, many network protocols are designed this way (Ethernet's exponential backoff after a collision is the classic case). But it works only with high probability. Is there a non-randomized solution?

Attempt 4: a global mutex. Protect the whole "take two, eat, put two" sequence with one more semaphore:

```
semaphore mutex = 1;
semaphore chopsticks[N];  // all initialized to 1

1  void philosopher(int i) {
2    for (;;) {
3      think();
4      down(&mutex);
5      take_chopstick(i);
6      take_chopstick((i + 1) % N);
7      eat();                          // critical section
8      put_chopstick((i + 1) % N);
9      put_chopstick(i);
10     up(&mutex);
11   }
12 }
```

Any deadlock? No: only one philosopher at a time ever touches the chopsticks. Any starvation? No, provided the semaphore wakes its waiters fairly. This solution is theoretically correct, but its performance is very bad: with the global mutex there is no parallelism, and only one philosopher can be eating at any time, although five chopsticks are enough for two.

Attempt 5: one semaphore per philosopher. The issues so far: modeling each chopstick as a semaphore is intuitive, but it does not work, since we are afraid to `down()` a chopstick, which may lead to a deadlock. The real issue is that *philosophers* need to eat, not chopsticks. So instead of worrying about chopsticks, we should guarantee: when a philosopher is eating, their left and right neighbors cannot eat.

```
int state[N];         // THINKING, HUNGRY, or EATING
semaphore mutex = 1;  // protects state[]
```

```
semaphore sem[N];     // all initialized to 0

void philosopher(int i) {
  for (;;) {
    think();
    take_chopsticks(i);  // section entry: block until I take both chopsticks
    eat();               // critical section
    put_chopsticks(i);   // section exit
  }
}

void take_chopsticks(int i) {
  down(&mutex);
  state[i] = HUNGRY;
  test(i);        // try to take both chopsticks
  up(&mutex);
  down(&sem[i]);  // block if cannot take chopsticks
}

void put_chopsticks(int i) {
  down(&mutex);
  state[i] = THINKING;
  test(LEFT);     // see if left neighbor can now eat
  test(RIGHT);    // see if right neighbor can now eat
  up(&mutex);
}

void test(int i) {
  if (state[i] == HUNGRY &&
      state[LEFT] != EATING &&
      state[RIGHT] != EATING) {
    state[i] = EATING;
    up(&sem[i]);  // take both chopsticks
  }
}
```

Here `LEFT` and `RIGHT` are the neighbors of the `i` in scope, `(i + N - 1) % N` and `(i + 1) % N`. How it works:

- `sem[i]` starts at 0; it is not a lock but a "you may eat now" signal for philosopher i. `test(i)` grants it by setting `state[i] = EATING` and `up(&sem[i])`.
- A hungry philosopher calls `test` on itself. If both neighbors are not eating, it grants itself permission, and the `down(&sem[i])` at the end of `take_chopsticks` returns immediately; otherwise that `down` blocks.
- A philosopher who finishes eating calls `test` on both neighbors, waking a neighbor who is hungry and can now eat.
- `mutex` protects `state[]`, and is always released before blocking on `sem[i]`: the lesson of the producer-consumer deadlock.

Only philosophers are modeled, not chopsticks.

Example 101 (A Run of Attempt 5)**.** All five philosophers start `THINKING` (after slides 225–232).

Event	P0	P1	P2	P3	P4	
start	T	T	T	T	T	
P0: `take_chopsticks(0)`	E	T	T	T	T	P4 and P1 are not eating
P1: `take_chopsticks(1)`	E	H	T	T	T	blocked on `mutex` first, then on `sem[1]`
P3: `take_chopsticks(3)`	E	H	T	E	T	P2 and P4 are not eating
P0: `put_chopsticks(0)`	T	E	T	E	T	`test(4)`: P4 not hungry; `test(1)`: P1 can eat

(T = `THINKING`, H = `HUNGRY`, E = `EATING`.) P1 called `take_chopsticks(1)` while P0 was inside its own `take_chopsticks` holding the mutex, so P1 first blocked on `mutex`. Once unblocked, it found P0 eating, so `test(1)` failed and P1 blocked on `sem[1]`. When P0 finished, its `test(1)` set P1 to `EATING` and uped `sem[1]`, unblocking P1. P1 and P3 now eat in parallel.

Key Point 102 (Dining Philosophers, Summary)

The problem is once again about mutual exclusion and synchronization: multiple processes compete for multiple resources, and each process needs two distinct resources to enter its critical section.

- The random backoff approach is adopted in many network protocols.
- The global mutex approach is deadlock-free and starvation-free, but only one philosopher can be eating at any instant.
- The final solution is deadlock-free and allows the maximum parallelism for any number of philosophers. However, a philosopher can still starve to death: if P0 and P2 take turns eating so that one of them is always eating, P1's `test` never succeeds.

2.10.3 The Readers and Writers Problem

Definition 103 (Readers and Writers Problem)

Processes access a database. Multiple processes are allowed to *read* the database at the same time, but if a process is *writing* the database, no other process may access it. How should the readers and writers be programmed?

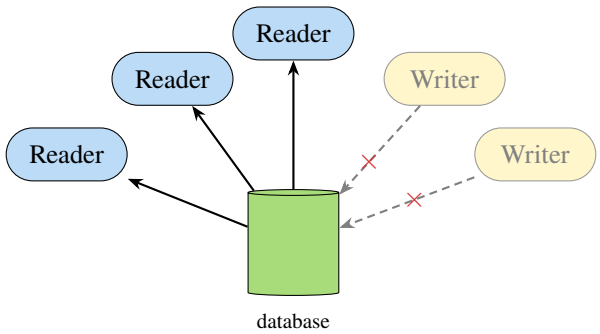


Figure 2.30: Readers may share the database; writers are excluded while anyone reads (after slides 235–236).

The requirements are:

- *mutual exclusion*: the database is a shared resource;

- *synchronization*: while a reader is reading, other readers may read but no writer may write; while a writer is writing, no reader or writer may access the database;
- *concurrency*: concurrent access from multiple readers should be allowed.

Attempt 1: a mutex for the database. Treat every access as a critical section protected by `semaphore db = 1`: each reader and each writer does `down(&db)` before accessing the database and `up(&db)` after. No two processes can be in their critical sections at the same time, so it is correct, but it violates the concurrency requirement: two readers cannot read together.

Attempt 2: allow concurrent readers. Readers do not need to exclude one another; but as long as there is a reader in the system, writers must be excluded. So the *first* reader that comes in should `down(&db)`, and the *last* reader that leaves should `up(&db)`. We just need to count the readers.

```
semaphore db = 1;      // controls access to the database
semaphore mutex = 1;   // controls access to "reader_count"
int reader_count = 0;  // # of processes reading or wanting to read

void reader() {
  for (;;) {
    down(&mutex);
    if (++reader_count == 1)
      down(&db);  // the first reader locks db
    up(&mutex);
    data = read_database();  // critical section
    down(&mutex);
    if (--reader_count == 0)
      up(&db);    // the last reader unlocks db
    up(&mutex);
    consume_data(data);
  }
}

void writer() {
  for (;;) {
    data = produce_data();
    down(&db);             // locks db
    write_database(data);  // critical section
    up(&db);               // unlocks db
  }
}
```

`reader_count` is itself shared, so it needs its own `mutex`. If a writer holds `db`, the first reader blocks on `down(&db)` while holding `mutex`, and later readers block on `mutex`; when the writer finishes, they all get in. Holding `mutex` while blocking is safe here because the writer never touches `mutex`, so there is no circular wait.

This solution meets all the requirements, but it gives readers a higher priority than writers: once one reader is in, new readers can keep joining. As long as there is a steady supply of readers, `reader_count` never drops to 0, and the writers starve.

2.10.4 The Sleeping Barber Problem

Definition 104 (Sleeping Barber Problem)

A barber shop has one barber, one barber chair, and n chairs for waiting customers. If there are no customers, the barber falls asleep in the barber chair. When a customer arrives, they wake up the barber. If more customers arrive while the barber is busy, they wait as long as there are empty chairs, and leave the shop if all chairs are full. How should the barber and the customers be programmed?

The problem models queueing situations in the *client-server model*. There is one long-running server process (the barber) and many transient client processes (the customers). The shared resources are the server itself, which is shared by all clients and serves one at a time, and a limited wait queue of n chairs; when the queue is full, new clients are dropped. We need mutual exclusion and proper synchronization.

```
semaphore barbers = 0;     // # of barbers ready to cut hair
semaphore customers = 0;   // # of customers waiting for service (not being cut)
int waiting = 0;           // same as above (there's no way to read a semaphore's value)
semaphore mutex = 1;       // controls access to "waiting"

void barber() {
  for (;;) {
    down(&customers);  // sleep if no customer
    down(&mutex);
    --waiting;
    up(&barbers);      // a barber is ready to serve
    up(&mutex);
    cut_hair();        // outside critical section
  }
}

void customer() {
  down(&mutex);
  if (waiting < NUM_CHAIRS) {
    ++waiting;
    up(&customers);  // wake up barber if needed
    up(&mutex);
    down(&barbers);  // wait if no barber is free
    get_haircut();   // outside critical section
  } else {
    up(&mutex);      // leave if the shop is full
  }
}
```

- `customers` counts the waiting customers; the barber sleeps on it when it is 0. Each arriving customer ups it, waking the barber if needed.
- `barbers` counts barbers that are ready to cut hair. It starts at 0, not 1, because the barber is *not* ready until it has picked a customer: the barber ups it once per haircut, after taking a customer from the waiting count, and the customer blocks on it until then. Starting at 1 would hand out one permit too many: the first customer would pass without the barber having picked them, and the barber's `up()` for that customer would then let a second customer into `get_haircut()` while the first is still in the chair.

- `waiting` duplicates the value of `customers`, because there is no way to read a semaphore's value, and a customer must check whether a chair is free before deciding to wait. `mutex` protects it.

The solution is free of data races and deadlocks. Can a customer starve? Every customer who sits down is counted in `customers`, and the barber serves one per `up(&barbers)`, so the shop as a whole keeps serving. But which blocked customer the `up(&barbers)` wakes is up to the semaphore. If it is not FIFO, a customer who sat down early can keep being passed over by later arrivals; with a FIFO semaphore, no one starves.

2.10.5 Summary

Key Point 105 (IPC Problems)

IPC problems involve one or more shared resources and multiple processes that must be synchronized. A good solution needs to

- guarantee mutual exclusion;
- guarantee proper synchronization among the processes;
- be deadlock-free;
- be starvation-free.

2.11 Threads

So far every process has had exactly one flow of execution. A thread separates that flow from the rest of the process, so that one process can run several flows at once.

Definition 106 (Thread)

A *thread* is an execution entity within a process. A *single-threaded* process has one; a *multithreaded* process has more than one, all running in the same address space.

A word processor, for example, may have one thread interacting with the user, one formatting text in the background, and one handling automatic backups.

2.11.1 What a Thread Owns

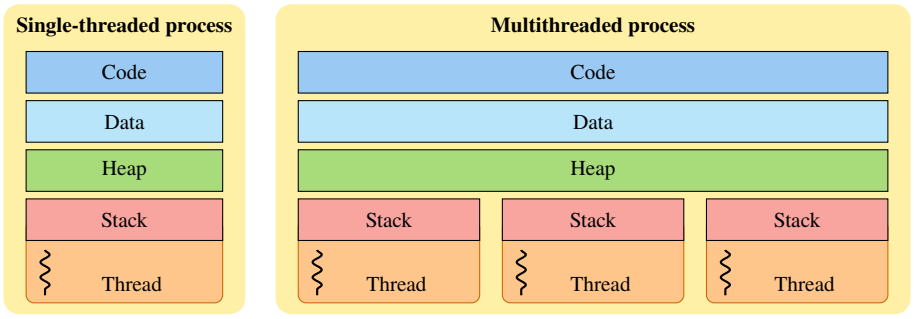


Figure 2.31: The threads of a process share its code, data, and heap; each thread has its own stack (after slides 248–250).

Figure 2.31 shows what is shared and what is not.

- *Code.* All threads share the same code. A new thread starts in a specific *thread function*, which can call other functions and make system calls, but it never returns to a caller: when it finishes, the thread ends.
- *Data and heap.* All threads share the same global variables and dynamically allocated memory. So a multithreaded process has race conditions too: `counter++` on a global variable from two threads goes wrong in exactly the way two processes went wrong on shared memory in Section 2.8.
- *Stack.* Each thread has its own stack for its local variables. The stacks are still in the one shared address space, so a thread that knows an address on another thread's stack can read and write it; nothing but convention keeps it out.

2.11.2 Why Threads?

- *Multitasking within a process.* One thread can wait for user input while another performs computation: when the thread reading from the keyboard blocks, the thread formatting text keeps running. This gives better performance and responsiveness.
- *Cheap.* Threads are 10–100× faster to create and destroy than processes, since there is no new address space to build.
- *Easy sharing.* Threads share the same address space, so sharing data needs no pipes or shared-memory setup: a pointer is enough.

2.11.3 Thread Models

Where are threads implemented? There are three models, distinguished by how many kernel-visible threads back the threads the program sees (Figure 2.32).

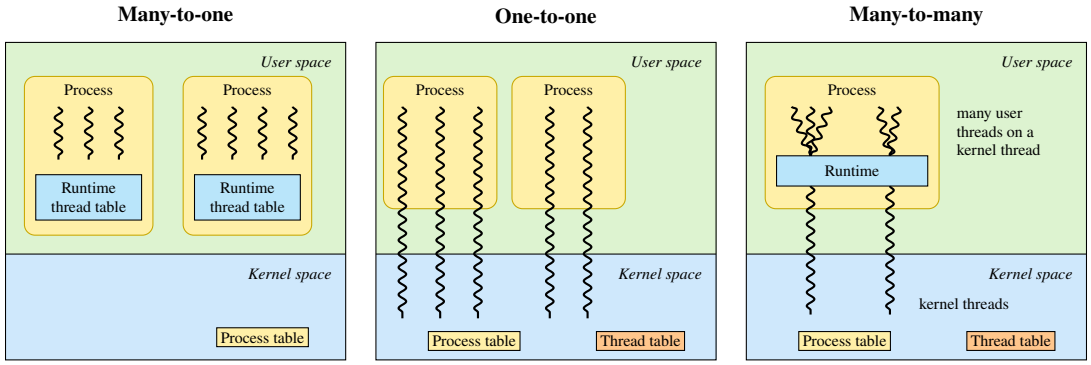


Figure 2.32: The three thread models; each wavy line is a thread. In the many-to-one model the kernel sees only processes (after slides 253–255).

Many-to-one: threads in user space. The threads library is entirely in user space, and the kernel knows nothing about threads (they are called *green threads*). Each process carries a runtime with its own thread table and switches among its threads by itself. This is how earlier operating systems did it.

- *Pros.* It does not need OS support. It is fast: a thread switch is a function call inside the runtime, with no trap, no context switch, and no cache flush.
- *Cons.* The kernel schedules the process as one unit, so when one thread invokes a blocking system call, all threads are blocked; likewise a page fault (later chapters) in one thread blocks the entire process. There is also no preemption among threads, because clock interrupts go to the kernel, not to the runtime: a thread runs until it gives up the CPU voluntarily.

One-to-one: threads in the kernel. The kernel manages all the threads: each thread is mapped to a kernel thread, and the kernel keeps a thread table next to its process table. Most modern operating systems do this; Linux's pthreads library (NPTL) is one-to-one.

- *Pros.* When a thread blocks, the kernel can schedule another thread, from the same process or a different one.
- *Cons.* Creating and destroying threads is more expensive, since each one is a system call. And the process-level mechanisms of this chapter now have to answer thread-level questions:
  - *What happens when a multithreaded process forks?* In POSIX, the child gets a copy of the whole address space but only one thread, the one that called `fork()`. Any mutex that another thread held at that moment is copied in its locked state, and nobody in the child will ever unlock it. So between `fork()` and `exec*()`, the child of a multithreaded process should call only async-signal-safe functions (`man 2 fork`).
  - *When a signal comes in, which thread handles it?* A signal sent to the process (e.g., by `kill()`) is delivered to an arbitrary thread that does not block it. A signal caused by a thread's own action, such as the `SIGSEGV` of a bad memory access, or one sent with `pthread_kill()`, goes to that particular thread (`man 7 signal`).

Many-to-many: a hybrid. A threads library in user space multiplexes user-level threads onto kernel threads. It is a two-level model, implemented in many language runtimes (Erlang, Go, Haskell, the JVM. . .).

- *Pros.* The best of both worlds: user threads are cheap to create and switch, and a blocking call ties up only one kernel thread. There is no restriction on the number of threads.
- *Cons.* It is more complex to implement: two schedulers, one in the runtime and one in the kernel, must cooperate.

2.11.4 POSIX Threads

The POSIX threads API (`man 7 pthreads`) has a thread counterpart for each process call of this chapter:

Description	Process	Thread
Process/thread identification	`pid_t`	`pthread_t`
Get process/thread ID	`getpid()`	`pthread_self()`
Create a new process/thread	`fork()`	`pthread_create()`
Terminate the calling process/thread	`exit()`	`pthread_exit()`
Wait for a specific process/thread to exit	`waitpid()`	`pthread_join()`
Send a signal to a process/thread	`kill()`	`pthread_kill()`
Release the CPU to let another process/thread run	`sched_yield()`	`pthread_yield()`

`pthread_yield()` is non-standard and deprecated in glibc; threads call `sched_yield()` as well.

Example 107 (`thread.c`)**.**

```
1  void *greet(void *name) {
2    printf("Hello, %s!\n", (char *)name);
3    pthread_exit(NULL);
4  }
5
6  int main() {
7    pthread_t tid;
8    pthread_create(&tid, NULL, greet, "world");
9    printf("I am still running...\n");
10   pthread_join(tid, NULL);
11 }
```

```
$ gcc -pthread thread.c -o thread
$ ./thread
I am still running...
Hello, world!
```

`pthread_create()` starts a new thread in the thread function `greet`, passing `"world"` through its `void *` argument (a thread function takes and returns a `void *`, so any data can be passed by pointer). Unlike `fork()`, it does not return twice: the main thread continues at line 9 while `greet` runs alongside it. `pthread_join()` then blocks the main thread until `greet` calls `pthread_exit()` (Figure 2.33). The order of the two lines of output is up to the scheduler; the run above is just one possibility. Without the join, `main` could return first, and returning from `main` exits the whole process, killing `greet` with it.



Figure 2.33: The run of `thread.c`: `pthread_join()` blocks the main thread until `greet` exits (after slide 257).

2.11.5 Synchronization in pthreads

Mutexes. A mutex object of type `pthread_mutex_t` is similar to a binary semaphore. It cannot be used as a counting semaphore.

Description	pthread mutex	Equivalent semaphore
Initialize	`pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;`	`semaphore mutex = 1;`
Lock (enter)	`pthread_mutex_lock(&mutex);`	`down(&mutex);`
Unlock (exit)	`pthread_mutex_unlock(&mutex);`	`up(&mutex);`

Semaphores. A POSIX semaphore of type `sem_t` is a counting semaphore, the `semaphore` of Section 2.9:

Description	POSIX semaphore	Equivalent "2250" semaphore
Initialize	`sem_t s; sem_init(&s, 0, initial_value);`	`semaphore s = initial_value;`
Down	`sem_wait(&s);`	`down(&s);`
Up	`sem_post(&s);`	`up(&s);`
Destroy	`sem_destroy(&s);`	—

The second argument of `sem_init()` is 0 for a semaphore shared by the threads of one process; a nonzero value shares it between processes, in which case the `sem_t` itself must live in shared memory (`man 3 sem_init`).

Condition variables. Semaphores are a convenient and effective synchronization mechanism, but using them incorrectly causes hard-to-detect timing errors, as the swapped `down()`s of the producer showed in Section 2.10. The pthread library provides another mechanism, which separates the lock from the waiting.

Definition 108 (Condition Variable)

A *condition variable* (type `pthread_cond_t`) lets a thread wait for a condition to become true. It is always used together with a mutex that protects the state the condition is about.

- `pthread_cond_wait(&cond, &mutex)`: the calling thread must hold `mutex`. When it is about to block, `mutex` is unlocked, automatically and atomically; when it is unblocked, `mutex` is locked again, automatically and atomically, before the call returns.
- `pthread_cond_signal(&cond)`: if some threads are blocked on `cond`, at least one of them is unblocked. If no thread is blocked on `cond`, the signal is lost.

It is initialized with `pthread_cond_t cond = PTHREAD_COND_INITIALIZER;`.

Releasing the mutex inside `pthread_cond_wait()` is what prevents the producer-consumer deadlock: a waiting thread never blocks while holding the mutex, so there is no *hold and wait* (Section 2.12). And a lost signal is the big difference from a semaphore, whose `up()` is remembered in its value even when nobody is waiting; a condition variable has no memory, so the condition itself must be kept in ordinary variables and checked under the mutex.

Example 109 (Producer-Consumer with pthreads)**.** The bounded buffer of Section 2.10 needs two conditions: the producer waits until the buffer is not full, and the consumer waits until it is not empty.

```
pthread_mutex_t mutex     = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  not_full  = PTHREAD_COND_INITIALIZER;
pthread_cond_t  not_empty = PTHREAD_COND_INITIALIZER;
```

```
1  void producer() {
2    int item;
3    for (;;) {
4      item = produce_item();
5      pthread_mutex_lock(&mutex);
6      while (count == BUFFER_SIZE)
7        pthread_cond_wait(&not_full, &mutex);
8      insert(item);  // increments count
9      pthread_cond_signal(&not_empty);
10     pthread_mutex_unlock(&mutex);
11   }
12 }
```

```
1  void consumer() {
2    int item;
3    for (;;) {
4      pthread_mutex_lock(&mutex);
5      while (count == 0)
6        pthread_cond_wait(&not_empty, &mutex);
7      item = remove();  // decrements count
8      pthread_cond_signal(&not_full);
9      pthread_mutex_unlock(&mutex);
10     consume_item(item);
11   }
12 }
```

The state is the shared `count`, read and written only under `mutex`. The producer checks the condition (line 6), waits if the buffer is full, inserts, and signals `not_empty` in case the consumer is waiting; the consumer is symmetric.

Why `while` rather than `if`? Being woken up is only a hint that the condition *may* hold. Between the signal and the moment the woken thread reacquires the mutex, another thread can take the mutex first and change `count` (say, a second consumer empties the buffer again), and POSIX also allows spurious wakeups. So the thread re-checks the condition after every wakeup.

Example 110 (Semaphores from Mutexes and Condition Variables)**.** A semaphore is a counter plus waiting, and that is exactly a variable, a mutex, and a condition variable:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  cond  = PTHREAD_COND_INITIALIZER;
int value;

void down() {
  pthread_mutex_lock(&mutex);
  while (value == 0)
    pthread_cond_wait(&cond, &mutex);
  --value;
  pthread_mutex_unlock(&mutex);
}

void up() {
```

```
    pthread_mutex_lock(&mutex);
    ++value;
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&mutex);
}
```

`up()` increments `value` even if no one is waiting, so the signal that may be lost is backed by the count that is not: a later `down()` sees `value > 0` and does not wait at all.

2.11.6 Thread Pools

Definition 111 (Thread Pool)

A *thread pool* is a design pattern for concurrent execution: a fixed set of worker threads is created at startup and waits for tasks to be dispatched from a task queue. Each idle worker takes the next task from the queue, runs it, and goes back to waiting.

If a program has a lot of short-lived tasks, creating and destroying a thread for each one is time-consuming. A pool pays that cost once, which increases performance and avoids latency when a task arrives. The task queue is a producer-consumer buffer (Figure 2.34): whoever submits tasks is the producer, and the workers are the consumers.

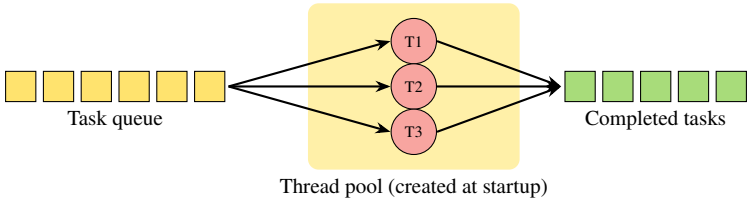


Figure 2.34: A thread pool: workers take tasks from the queue and run them (after slide 264).

2.11.7 Thread Safety

Some data structures and functions are designed for single-threaded execution. `strtok()`, for example, keeps a pointer into the string being parsed in a static variable between calls, so two threads tokenizing two strings at the same time overwrite each other's position. It is not *thread safe*.

Definition 112 (Reentrant Function)

A function is *reentrant* if multiple invocations can safely run concurrently: it may be interrupted in the middle of its execution and then safely be invoked again before its previous invocations complete. This requires keeping all state information in variables local to each invocation, rather than in static or global variables.

Making a function reentrant is one way to make it thread safe. `strtok_r()` is the reentrant version of `strtok()`: it takes an extra argument, a pointer owned by the caller, in which it keeps its context between successive calls that parse the same string.

```
char *save;  // the caller's own parsing state
for (char *tok = strtok_r(line, " ", &save); tok != NULL;
     tok = strtok_r(NULL, " ", &save))
  puts(tok);
```

2.11.8 Summary

Key Point 113 (Threads)

Multiple threads are a convenient way to get multitasking within a process and a simple shared-memory environment. The price is that you have to take care of mutual exclusion, synchronization, and thread safety yourself.

2.12 Deadlocks

Deadlocks have come up twice already: the producer with its two `down()`s swapped, and the dining philosophers each holding a left chopstick (Section 2.10). This section looks at them in general: when they can happen, and four ways of dealing with them. From now on the actors are called threads, but everything applies to processes as well.

2.12.1 Resource Allocation Graphs

The slides open with the restroom picture from Section 2.10: one person holds a stall and waits for paper, the other holds the paper and waits for the stall. A graph makes the same picture precise.

Definition 114 (Resource Allocation Graph)

A *resource allocation graph* has a node for each thread (a circle) and for each resource (a square). An edge from a resource to a thread means the thread is *holding* the resource; an edge from a thread to a resource means the thread is *requesting* it, i.e., blocked waiting for it.

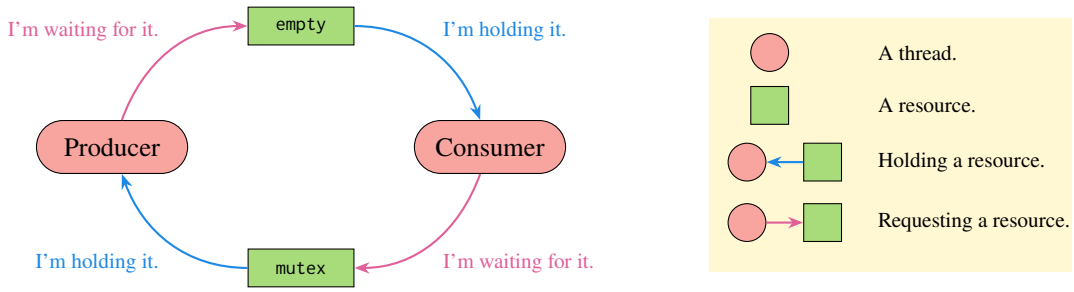


Figure 2.35: The producer-consumer deadlock of Figure 2.27 as a resource allocation graph: a cycle (after slide 269).

This is a *resource deadlock*: each thread holds a resource the other one needs, and the graph has a cycle.

Key Point 115 (Conditions for Resource Deadlocks)

A resource deadlock can occur only if all four of the following conditions hold:

1. *Mutual exclusion.* Each resource is either available or currently assigned to exactly one thread.
2. *Hold and wait.* Threads currently holding resources that were granted earlier can request new resources.
3. *No preemption.* Resources previously granted cannot be forcibly taken away from a thread; they must be explicitly released by the thread holding them.
4. *Circular wait.* There is a cycle of two or more threads, each of which is waiting for a resource held by the next member of the cycle.

There are four strategies for dealing with deadlocks: ignore them, prevent them by breaking one of the four conditions, detect them and recover, or avoid them by careful resource allocation. The rest of the section takes them in that order.

2.12.2 The Ostrich Algorithm

The most widely adopted approach is to stick your head in the sand and pretend there is no problem. This is not a joke. If

- the deadlock happens once in a blue moon,
- detecting or preventing it is expensive,
- you can resolve it by simply rebooting, and
- the system crashes for other reasons much more frequently,

why not just let it be? Engineering is a trade-off between the cost of a fix and the cost of the failure.

2.12.3 Prevention: Attacking the Four Conditions

If any one of the four conditions of Key Point 115 can never hold, deadlocks become structurally impossible. Attack them one at a time.

Attack 1: no mutual exclusion.

- *Method 1: make the data read-only*, so that threads can use it concurrently without locks.
- *Method 2: lock-free data structures*, built from powerful atomic hardware instructions instead of locks.
- *Method 3: read-copy-update*, below.

For lock-free data structures, test-and-set (Section 2.9) is not enough; the workhorse is *compare-and-swap* (CAS). Conceptually it executes the following code atomically: it writes `new` only if the value is still the one the caller expected.

```
bool compare_and_swap(int *ptr, int expected, int new) {
  // this code executes atomically
  if (*ptr != expected)
    return false;
  *ptr = new;
  return true;
}
```

Example 116 (Atomic Increment)**.** To execute `*ptr += amount` atomically, read the old value and try to install the new one; if another thread changed `*ptr` in between, the CAS fails and we retry with the fresh value.

```
void atomic_increment(int *ptr, int amount) {
  int old;
  do {
    old = *ptr;
  } while (!compare_and_swap(ptr, old, old + amount));
}
```

There is no lock, therefore no deadlock. Is there starvation? Yes, in principle: under heavy contention, a thread can lose the race every time, because some other thread's CAS keeps succeeding between its read and

its own CAS. The system as a whole always makes progress (some CAS succeeds), but an individual thread may not. That is the guarantee called *lock-free*, as opposed to the stronger *wait-free*.

Example 117 (Lock-Free Insertion into a Linked List)**.** Inserting a node at the head of a shared list:

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  n->next = head;
  head = n;
}
```

The race: two threads both read the same `head` into their `n->next`, then both write `head`. The second write wins, and the first thread's node is lost from the list. With CAS, the last two lines become "set `head` to `n` only if `head` is still what I linked `n` to", retried until it succeeds:

```
void insert(int value) {
  node_t *n = malloc(sizeof(node_t));
  assert(n != NULL);
  n->value = value;
  do {
    n->next = head;
  } while (compare_and_swap(&head, n->next, n) == 0);
}
```

(Here `compare_and_swap` works on pointers rather than `int`s.) Before `head` changes, the new node is private to its thread, so preparing it needs no protection at all; only the single pointer update has to be atomic.

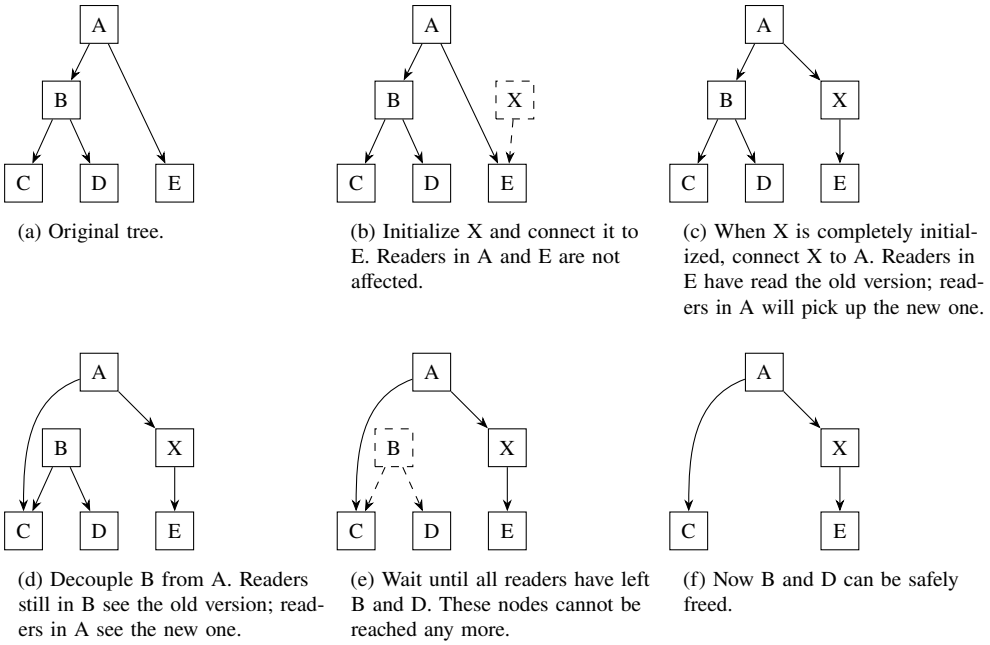
Definition 118 (Read-Copy-Update)

Read-copy-update (RCU) allows a writer to update a data structure while other threads are still reading it. Every update is arranged so that a reader sees and traverses either the old version or the new version, never some mash-up of the two. RCU decouples the *removal* phase of an update, after which new readers can no longer reach the old data, from the *reclamation* phase, in which the old data is freed once no reader can still be using it.

In Figure 2.36, each step changes one pointer, and a single pointer store is atomic, so a reader walking down from A follows either the old pointer or the new one. Insertion builds the new node completely before publishing it with one store (b, c). Removal unpublishes B with one store (d); this is the removal phase. B and D cannot be freed yet, because a reader that was already in B is still allowed to finish its traversal. The writer waits for a *grace period*, until every reader that might have seen B is done (e), and only then reclaims B and D (f). Readers take no locks at all.

Attack 2: no hold and wait.

- *Method 1: request all resources at once, before starting execution.* A thread that holds everything it needs never waits. But a thread may not know what resources it will need until it has started running, and resources are held for longer than needed, which decreases concurrency. Nevertheless, some mainframe batch systems use this approach.



(a) Original tree.

(b) Initialize X and connect it to E. Readers in A and E are not affected.

(c) When X is completely initialized, connect X to A. Readers in E have read the old version; readers in A will pick up the new one.

(d) Decouple B from A. Readers still in B see the old version; readers in A see the new one.

(e) Wait until all readers have left B and D. These nodes cannot be reached any more.

(f) Now B and D can be safely freed.

Figure 2.36: RCU: inserting a node X (a–c), then removing B (d–f) (after slides 276–277).

- *Method 2: release all resources before requesting a new one.* A thread that holds nothing while it waits cannot be part of a cycle. `pthread_cond_wait()` (Section 2.11) follows this idea: it releases the mutex before blocking.

Attack 3: no "no preemption". Technically, we cannot forcibly take away a resource that a thread already holds. In practice, a thread can preempt its own ownership in a graceful way: instead of blocking on the second resource, it *tries* to take it, and if that fails, it gives back the first. This is *trylock* (`pthread_mutex_trylock()` in pthreads), and it is exactly Attempts 2 and 3 of the dining philosophers in Section 2.10:

- with a fixed backoff (`sleep(1)`), philosophers running in lockstep take, fail, put back, and sleep forever together: a *livelock*;
- with a random backoff (`sleep(random())`), the lockstep breaks with high probability, which is good enough for most applications and is how many network protocols are designed.

Attack 4: no circular wait. This is probably the most practical and frequently used approach. Give all the resources a total order (a global numbering), and let a thread request only resources numbered higher than any it is already holding.

Key Point 119 (Resource Ordering Prevents Circular Wait)

Suppose threads $T_1, \ldots, T_k$ formed a cycle, where T_i holds resource r_i and waits for r_{i+1}, which T_{i+1} holds (indices mod k). Since T_i requests r_{i+1} while holding r_i, the ordering rule gives $r_i < r_{i+1}$ for every i. Going around the cycle, $r_1 < r_2 < \cdots < r_k < r_1$, a contradiction.

Example 120 (Ordered Chopsticks)**.** Attempt 1 of the dining philosophers (Figure 2.29) deadlocks because every philosopher takes chopstick i and then $(i+1) \bmod N$, and for philosopher $N-1$ that second chopstick

is 0, lower than the first: the order wraps around and closes the cycle. Take the lower-numbered chopstick first instead:

```
void philosopher(int i) {
  int first = min(i, (i + 1) % N);
  int second = max(i, (i + 1) % N);
  for (;;) {
    think();
    take_chopstick(first);
    take_chopstick(second);
    eat();  // critical section
    put_chopstick(second);
    put_chopstick(first);
  }
}
```

Only philosopher $N - 1$ changes: it now reaches for chopstick 0 first, competing with philosopher 0. If all of them get hungry at once, one of those two loses the race for chopstick 0 and blocks holding nothing. That leaves a gap in the circle, so some philosopher can get both chopsticks and eat, and the rest follow as chopsticks are put down.

2.12.4 Detection and Recovery

Another strategy is to allow deadlocks to occur (occasionally), try to detect when this happens, and then take some action to recover after the fact.

Detection with one resource of each type. A deadlock detector runs periodically and builds the resource allocation graph. When there is exactly one resource of each type, the threads are deadlocked if and only if the graph has a cycle, and the deadlocked threads are those on the cycle.

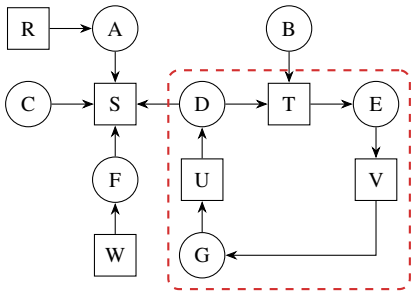


Figure 2.37: A resource allocation graph with one resource of each type. Threads D, E, and G are deadlocked on the cycle D → T → E → V → G → U → D (after slide 286).

How to find a cycle in a directed graph? Depth-first search. Mark each node as unvisited, on the current path, or finished. Start a DFS from every unvisited node; when the search follows an edge into a node that is *on the current path*, the path from that node back to itself is a cycle. When a node's edges are exhausted, mark it finished and backtrack; an edge into a finished node cannot close a cycle, since everything reachable from there has already been explored. Each node and edge is handled once, so the check is linear in the size of the graph. In Figure 2.37, a DFS from D goes D → T → E → V → G → U and then follows U → D back into the current path: deadlock among D, E, and G. A, C, and F are not deadlocked: nobody holds S, so

one of them gets it, finishes, and releases it for the next. B is not on the cycle, but it waits for T, which the deadlocked E holds, so B will never proceed either.

(With several resources of each type, e.g., three identical printers, a cycle is necessary for a deadlock but no longer sufficient, and detection needs a matrix-based algorithm instead.)

Recovery. None of the ways to recover is especially attractive.

- *Preemption.* Temporarily take a resource away from its owner and give it to another thread. Whether this is possible depends highly on the nature of the resource, and it often requires manual intervention; recovering this way is often difficult or impossible.
- *Rollback.* Checkpoint threads periodically by writing their states to a file, so that they can be restarted later. A thread that owns a needed resource is rolled back to a checkpoint from before it acquired that resource.
- *Killing.* Kill threads until the cycle is broken.

2.12.5 Avoidance

Can the system avoid deadlocks by tightly controlling resource allocation, granting each request only when it is safe to do so? The *banker's algorithm* (Dijkstra) does this: it grants a request only if, afterwards, there is still some order in which every thread can obtain its maximum demand and finish. It is very nice in theory, but essentially useless in practice, because it requires information about future requests (each thread's maximum demand), which is rarely available in advance.

2.12.6 Summary

Key Point 121 (Dealing with Deadlocks)

Strategy	Idea
Ostrich	ignore deadlocks when they are rare and cheap to fix by rebooting
Prevention	make one of the four conditions impossible: lock-free structures or RCU (mutual exclusion), all-at-once requests (hold and wait), trylock with backoff (no preemption), resource ordering (circular wait)
Detection and recovery	find cycles in the resource allocation graph; then preempt, roll back, or kill
Avoidance	grant only safe requests (banker's algorithm); needs future knowledge

2.12.7 Process Management in Retrospect

This section ends the deck. Its questions, and where they were answered:

- What is a process, and how is a program related to a process? (Section 2.1)

- How are processes created? (Sections 2.2 to 2.4)
- How are processes organized? (Sections 2.5 and 2.6)
- How are processes scheduled? (Section 2.7)
- How do processes cooperate? (Section 2.8)
- What problems occur when processes cooperate? (Sections 2.9, 2.10 and 2.12)
- What are threads, and how are they programmed? (Section 2.11)

Appendix A

System Call Reference

The process-management and IPC calls used in these notes, with the section where each is discussed.

Call	What it does	See
`getpid(), getppid()`	PID of the caller, PID of its parent	Section 2.2
`fork()`	clone the caller; returns the child's PID in the parent, 0 in the child, -1 on failure; memory is shared copy-on-write	Sections 2.2 and 2.4
`execve(), exec*()`	replace the program the process runs; does not return on success; keeps the PID and the fd table	Sections 2.2 and 2.4
`exit()`	free memory, close files, leave a zombie, send `SIGCHLD` to the parent	Section 2.4
`wait(), waitpid()`	block until a child terminates and reap it; -1 if there are no children	Sections 2.2 and 2.4
`dup2(old, new)`	make descriptor `new` refer to the open file of `old`; the basis of redirection	Section 2.3
`kill(pid, sig)`	send a signal to a process; `raise(sig)` sends it to the caller	Section 2.5
`signal(sig, h)`	install a handler and return the old one; not possible for `SIGKILL` and `SIGSTOP`	Section 2.5
`pause()`	block until a handled or terminating signal arrives	Section 2.5
`alarm(n), setitimer()`	one-time or periodic timer delivering `SIGALRM`	Section 2.5
`sched_yield()`	give up the CPU and go to the back of the run queue	Sections 2.7 and 2.9
`pipe(fd)`	create a pipe: `fd[0]` is the read end, `fd[1]` the write end; reads block on empty, writes on full	Section 2.8
`mmap()`	map memory; `MAP_SHARED | MAP_ANONYMOUS` before `fork()` gives shared memory	Section 2.8
`sem_wait(), sem_post()`	POSIX semaphore `down()` and `up()`; `sem_init()` with `pshared` $\neq 0$ for processes	Section 2.9

Bibliography

[1] R. E. Bryant and D. R. O'Hallaron. *Computer Systems: A Programmer's Perspective*, 3rd edition. Pearson, 2015.

[2] A. S. Tanenbaum and H. Bos. *Modern Operating Systems*, 5th edition. Pearson, 2022.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne. *Operating System Concepts*, 10th edition. Wiley, 2018.

[4] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018.